

Allgemeine Technische Dokumentation

„WEATHERLAB“

Teilnehmer:

Fabian Seelig, Julian Weißböck, Andrea Roithmeier
Matthias Winklehner, David Sageder

Inhaltsverzeichnis

1. Projektüberblick 3
1.1 Verwendete Hardware 3
1.2 Verwendete Software 3
1.3 Aufbau des Systems 3
1.4 Adressen im Überblick 4

2. Installation des Betriebssystems 5
2.1 Modell und Betriebssystem auswählen 5
2.2 Speicher auswählen 5
2.3 Voreinstellungen vornehmen 6
2.4 Image schreiben 6
2.5 Erster Start 7

3. Netzwerkkonfiguration – Raspberry Pi als Access Point 8
3.1 Installierte Dienste 8
3.2 NetworkManager-Konfiguration 8
3.3 DNS- / DHCP-Konfiguration 9
3.4 Website-Namensauflösung 9
3.5 WLAN-Konfiguration 10
3.6 Standardkonfiguration von hostapd 11
3.7 Startskript für das WLAN (startupWIFI.sh) 11
3.8 Automatischer Start des Skripts 12
3.9 Internet-Weitergabe ins WLAN 13

4. Sensorik – BME680 14
4.1 Der Sensor 14
4.2 Verkabelung 14
4.3 I2C aktivieren 14
4.4 Python-Bibliothek 15
4.5 Konfiguration des Sensors 16

5. Status-LEDs für die Luftqualität 17
5.1 Zweck 17
5.2 Verkabelung 17
5.3 Benötigte Bibliotheken 18
5.4 Test der LEDs 18
5.5 Einbindung in das Programm 18

6. Webanwendung 19
6.1 Aufbau des Projektordners 19
6.2 Die Datei config.py 19
6.3 Die Datei app.py 20
6.4 Schnittstellen 21
6.5 Datenbank 21

7. Berechnete Werte 22
7.1 Luftqualität 22
7.2 Luftdruck und Tendenz 22
7.3 Lüftungsempfehlung 23
7.4 Weitere abgeleitete Werte 23

8. Weboberfläche 25
8.1 Aufbau 25
8.2 Die fünf Seiten 25
8.3 Gestaltung 26
8.4 Kartenmaterial 27
8.5 Aktualisierung der Werte 28
8.6 Zusatzfunktionen für den Dauerbetrieb 28

9. Webserver und Autostart 29
9.1 Dienst für die Webanwendung 29
9.2 Apache als Reverse Proxy 29
9.3 Kiosk-Modus am Touch Display 30
9.4 Übersicht der Dienste 31

10. Telegram-Bot 32
10.1 Bot bei Telegram anlegen 32
10.2 Aufbau von telegram_bot.py 33
10.3 Alarme und Tagesbericht 34
10.4 Empfänger freischalten 36
10.5 Dienst telegram-bot 36
10.6 Einrichtung mit telegram-setup.sh 37
10.7 Test ohne echte Telegram-Verbindung 38

11. Betrieb, Wartung und Fehlerbehebung 39
11.1 Änderungen übernehmen 39
11.2 Nützliche Befehle 39
11.3 Sicherung 39
11.4 Aufgetretene Probleme und ihre Lösung 39
11.5 Hinweise für den Dauerbetrieb 40
11.6 Fazit 40

12. Anhang: Quellcode app.py 42
12.1 Kopf und Einstellungen 42
12.2 Gemeinsame Datenspeicher 43
12.3 Datenbank 43
12.4 Auswertung des Verlaufs 44
12.5 Status-LEDs 45
12.6 Sensor 46
12.7 Abfrage bei OpenWeather 47
12.8 Systemwerte des Raspberry Pi 49
12.9 Schnittstellen und Programmstart 49
12.10 Wiederkehrende Muster im Code 51

1. Projektüberblick

In diesem Projekt haben wir mit einem Raspberry Pi 5 eine eigene Wetterstation gebaut. Der Raspberry Pi übernimmt dabei gleich zwei Aufgaben: Er spannt ein eigenes WLAN auf und stellt gleichzeitig eine Website bereit, auf der die Messwerte angezeigt werden.

Die Innenwerte liefert ein BME680-Sensor, der direkt am Raspberry Pi angeschlossen ist. Die Außenwerte holen wir uns über das Internet von OpenWeather. Damit lassen sich Innen- und Außenklima direkt miteinander vergleichen.

Zusätzlich meldet ein Telegram-Bot Grenzwertüberschreitungen und Wetterwarnungen aufs Handy und schickt jeden Morgen einen Tagesbericht (Kapitel 10).

1.1 VERWENDETE HARDWARE

BAUTEIL	BESCHREIBUNG
Raspberry Pi 5	Einplatinencomputer, läuft mit Raspberry Pi OS 64 Bit
Pi Touch Display 2 (7")	zeigt die Website dauerhaft im Vollbild an
BME680 (CJMCU-680)	misst Temperatur, Luftfeuchte, Luftdruck und Luftqualität
3 LEDs (grün, gelb, rot)	zeigen die Luftqualität auf einen Blick an
Steckbrett und Jumperkabel	für die Verkabelung von Sensor und LEDs
LAN-Kabel	verbindet den Raspberry Pi mit dem Internet

1.2 VERWENDETE SOFTWARE

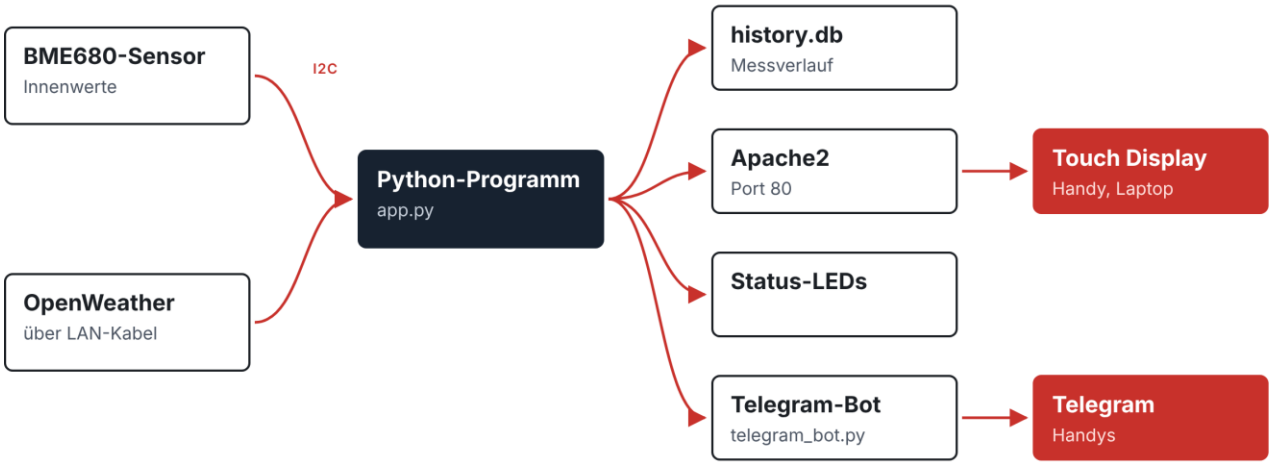
SOFTWARE	AUFGABE
hostapd	sendet das eigene WLAN aus
dnsmasq	vergibt IP-Adressen und löst Namen auf
Python 3 mit Flask	liest den Sensor aus und stellt die Daten bereit
SQLite	speichert den Messverlauf in einer Datei
Apache2	liefert die Website über Port 80 aus
Chromium	zeigt die Website auf dem Touch Display an
Telegram-Bot-API	verschickt Alarme, Tagesbericht und Antworten aufs Handy

1.3 AUFBAU DES SYSTEMS

Die einzelnen Teile arbeiten folgendermaßen zusammen: Der Sensor ist über I2C mit dem Raspberry Pi verbunden und wird von unserem Python-Programm jede Sekunde ausgelesen. Dasselbe Programm fragt im Hintergrund alle 60 Sekunden das aktuelle Wetter bei OpenWeather ab und speichert jede Minute einen Messpunkt in einer Datenbank.

Apache nimmt die Anfragen der Geräte entgegen und leitet sie intern an unser Programm weiter. Dadurch ist die Website unter der normalen Adresse ohne Portangabe erreichbar.

Der Telegram-Bot läuft als eigenes Programm neben der Webanwendung. Er holt die Messwerte über dieselben Schnittstellen wie die Website und schickt Nachrichten über das Internet an die Handys der Empfänger.



1.4 ADRESSEN IM ÜBERBLICK

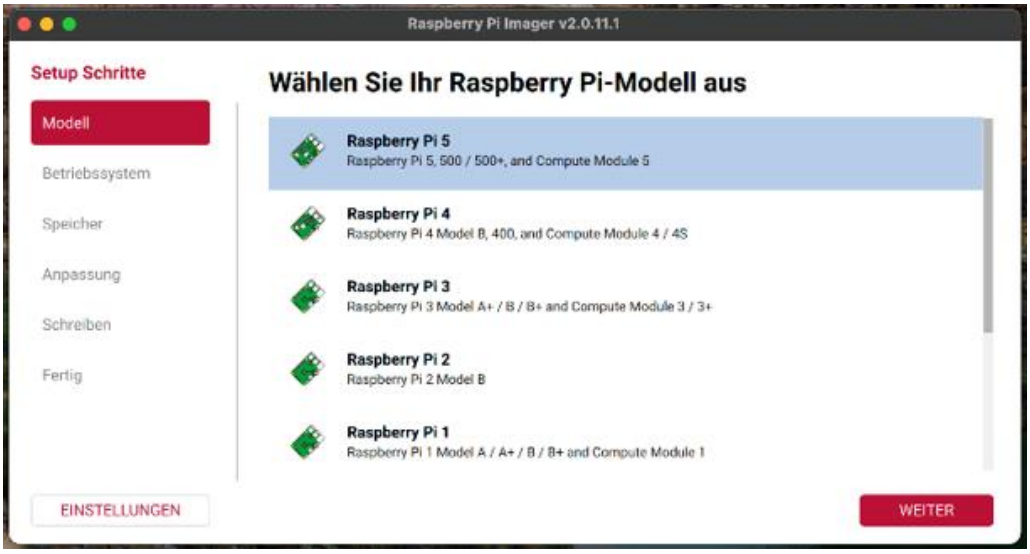
BEZEICHNUNG	WERT
Hostname	weatherlab
Benutzername	weatherlabadmin
WLAN-Name (SSID)	WeatherLab
IP-Adresse des Raspberry Pi (AP-Gateway)	192.168.10.1
Adressbereich für Geräte (AP)	192.168.10.10 bis 192.168.10.20
Adresse der Website	http://192.168.10.1/
Projektordner	/home/weatherlabadmin/pi-dashboard

2. Installation des Betriebssystems

Als erstes muss man für den Raspberry Pi den Raspberry Pi Imager herunterladen. Das Programm findet man unter https://www.raspberrypi.com/software/. Die heruntergeladene Datei wird installiert, danach kann die SD-Karte vorbereitet werden.

2.1 MODELL UND BETRIEBSSYSTEM AUSWÄHLEN

Im Imager wählen wir zuerst das Modell **Raspberry Pi 5** aus. Danach wird das Betriebssystem ausgewählt, wir nehmen **Raspberry Pi OS (64-bit)**.





2.2 SPEICHER AUSWÄHLEN

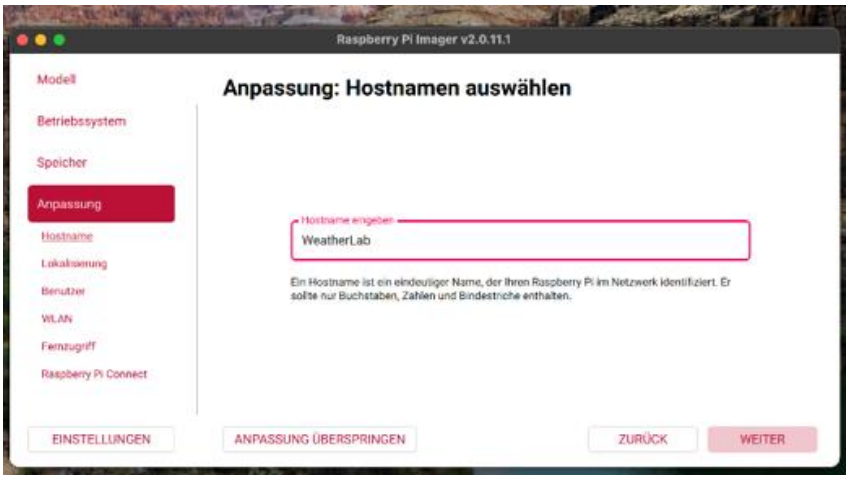
Als Speichermedium wird die eingesteckte SD-Karte ausgewählt. Wir verwenden eine Karte mit 128 GB.

2.3 VOREINSTELLUNGEN VORNEHMEN

Über die Schaltfläche *Einstellungen* lassen sich schon vor dem Schreiben wichtige Dinge festlegen. Das spart später viel Arbeit, weil man den Raspberry Pi dann gleich über das Netzwerk erreichen kann.

EINSTELLUNG	GEWÄHLTER WERT
Hostname	weatherlab
Benutzername	weatherlabadmin
Passwort	frei gewählt
Hauptstadt	Vienna (Austria)
Zeitzone	Europe/Vienna
Tastaturlayout	at
SSH	aktiviert, Passwort zur Authentifizierung

Die Einstellung für SSH ist besonders wichtig. Ohne sie könnten wir uns später nicht vom Windows-PC aus auf den Raspberry Pi verbinden.





2.4 IMAGE SCHREIBEN

Nach der Bestätigung werden die Einstellungen angezeigt und das Image auf die SD-Karte geschrieben. Der Vorgang dauert einige Minuten, währenddessen darf das Speichermedium nicht getrennt werden.



2.5 ERSTER START

Die fertige SD-Karte wird in den Raspberry Pi gesteckt, das LAN-Kabel angeschlossen und das Gerät eingeschaltet. Nach etwa einer Minute kann man sich per SSH verbinden:

```
ssh weatherlabadmin@weatherlab.local
```

Danach bringen wir das System auf den aktuellen Stand:

```
sudo apt update
sudo apt full-upgrade -y
sudo reboot
```

Der erste Durchlauf kann einige Minuten dauern. Nach dem Neustart verbindet man sich erneut per SSH.

3. Netzwerkkonfiguration – Raspberry Pi als Access Point

3.1 INSTALLIERTE DIENSTE

Für den Betrieb des WeatherLab Access Points wurden auf dem Raspberry Pi mehrere Dienste installiert und eingerichtet. Dazu gehören:

- hostapd (sendet das WLAN aus)
- dnsmasq (DHCP- und DNS-Server)
- iptables-persistent (speichert die Regeln für die Internet-Weitergabe)

Installiert wurden sie mit:

```
sudo apt install -y hostapd dnsmasq iptables-persistent
sudo systemctl stop hostapd
sudo systemctl stop dnsmasq
sudo systemctl unmask hostapd
```

Der Befehl `unmask` ist notwendig, weil hostapd nach der Installation gesperrt ist und sonst mit der Meldung *Unit hostapd.service is masked* nicht startet.

In den folgenden Punkten werden die jeweilige Konfiguration der Dienste sowie die dazugehörigen Dateipfade näher erläutert.

3.2 NETWORKMANAGER-KONFIGURATION

Diese Konfiguration befindet sich unter dem Pfad **/etc/NetworkManager/conf.d/unmanaged.conf**.

Aktuelle Versionen von Raspberry Pi OS verwalten alle Netzwerkkarten über den NetworkManager. Dieser würde hostapd die WLAN-Karte wegnehmen, deshalb muss das Interface wlan0 ausgenommen werden.

```
[keyfile]
unmanaged-devices=interface-name:wlan0
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

- **[keyfile]** Abschnittsbezeichnung, die zwingend in eckigen Klammern stehen muss.
- **unmanaged-devices=interface-name:wlan0** Legt fest, dass der NetworkManager das Interface wlan0 nicht verwaltet. Der Doppelpunkt vor wlan0 ist Pflicht, bei einem Gleichheitszeichen startet der Dienst nicht mehr.

Ob die Einstellung greift, lässt sich so prüfen:

```
nmcli device status
```

Bei wlan0 muss in der Spalte STATE das Wort **unmanaged** stehen.

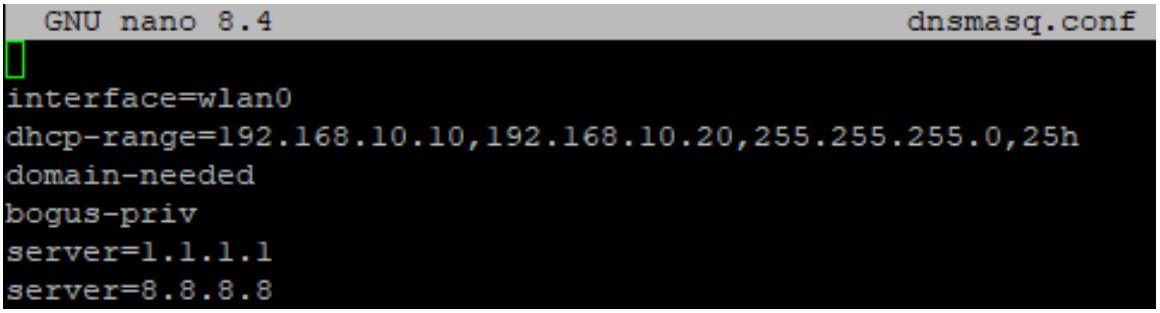
3.3 DNS- / DHCP-KONFIGURATION

Die Konfigurationsdatei befindet sich unter folgendem Pfad: **/etc/dnsmasq.conf**.

```
interface=wlan0
dhcp-range=192.168.10.10,192.168.10.20,255.255.255.0,24h
domain-needed
bogus-priv
server=1.1.1.1
server=8.8.8.8
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

- **interface=wlan0** Legt fest, dass dnsmasq auf dem Interface wlan0 arbeitet. Über dieses Interface wird der DHCP-Adressbereich bereitgestellt. Am LAN-Kabel ist der Dienst nicht aktiv.
- **dhcp-range=192.168.10.10,192.168.10.20,255.255.255.0,24h** Die konfigurierte DHCP-Range liegt im Bereich 192.168.10.10 bis 192.168.10.20. Aus diesem Bereich werden IP-Adressen automatisch an verbundene WLAN-Clients vergeben, die Gültigkeitsdauer beträgt 24 Stunden.
- **domain-needed** Unvollständige Namen werden nicht an die DNS-Server im Internet weitergeleitet.
- **bogus-priv** Anfragen zu privaten Adressbereichen beantwortet dnsmasq selbst.
- **server=1.1.1.1 / server=8.8.8.8** Diese DNS-Server werden an die Clients weitergegeben, damit im WLAN Namen aufgelöst werden können. Wichtig ist, dass hier nur erreichbare Adressen eingetragen sind. Ein nicht erreichbarer Eintrag führt dazu, dass sich Geräte nur sehr langsam verbinden.



3.4 WEBSITE-NAMENSAUFLÖSUNG

Die folgende Konfiguration befindet sich unter dem Pfad **/etc/dnsmasq.d/localips.conf**.

```
address=/weatherlab.lan/192.168.10.1
```

In dieser Datei werden benutzerdefinierte DNS-Auflösungen definiert. Der Eintrag sorgt dafür, dass die Domain weatherlab.lan auf die IP-Adresse 192.168.10.1 aufgelöst wird. Dadurch muss man im Browser nicht die IP-Adresse eintippen, sondern kann den Namen verwenden.

3.5 WLAN-KONFIGURATION

Hier wurde das WLAN konfiguriert, das ausgestrahlt wird. Diese Konfigurationsdatei ist unter folgendem Pfad zu finden: **/etc/hostapd/hostapd.conf**

```
interface=wlan0
driver=nl80211
ssid=WeatherLab
hw_mode=g
channel=6
country_code=AT
auth_algs=1
wpa=2
wpa_passphrase=Projekt2.
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

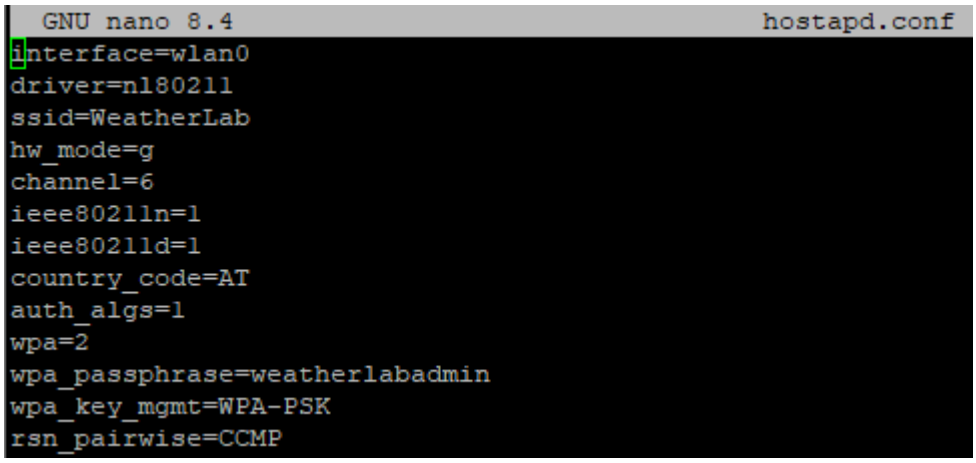
- **interface=wlan0** Legt das WLAN-Interface fest, das als Access Point verwendet wird.
- **driver=nl80211** Definiert den Linux-WLAN-Treiber bzw. die Schnittstelle zur WLAN-Hardware.
- **ssid=WeatherLab** Name des WLAN-Netzwerks (SSID), der für Clients sichtbar ist.
- **hw_mode=g** Setzt den WLAN-Modus auf 802.11g im 2,4-GHz-Band.
- **channel=6** Bestimmt den verwendeten Funkkanal (hier Kanal 6).
- **country_code=AT** Legt den Ländercode (Österreich) für regulatorische Vorgaben fest.
- **auth_algs=1** Verwendet Open System Authentication.
- **wpa=2** Aktiviert WPA2-Verschlüsselung.
- **wpa_passphrase=Projekt2.** Passwort für den WLAN-Zugang (Pre-Shared Key), mindestens 8 Zeichen.
- **wpa_key_mgmt=WPA-PSK** Verwendet WPA mit Pre-Shared Key zur Authentifizierung.
- **rsn_pairwise=CCMP** Legt AES-basierte CCMP-Verschlüsselung für WPA2 fest.

Bei der Einrichtung hatten wir zunächst auch die Einträge `ieee80211n=1` und `ieee80211d=1` in der Datei. Beim Abtippen wurden diese Zeilen jedoch unvollständig übernommen, wodurch hostapd mit der Meldung *unknown configuration item* nicht mehr startete. Da es sich dabei nur um Zusatzoptionen für höhere Datenraten handelt, haben wir sie weggelassen.

Fehler in dieser Datei lassen sich am besten so finden:

```
sudo hostapd -dd /etc/hostapd/hostapd.conf
```

Die Ausgabe nennt die fehlerhafte Zeile samt Zeilennummer. Beendet wird der Test mit Strg+C.



3.6 STANDARDKONFIGURATION VON HOSTAPD

Die Datei **/etc/default/hostapd** enthält die Standardkonfiguration für den hostapd-Dienst, der das System als WLAN-Access-Point betreibt.

Wichtige Parameter:

- **DAEMON_CONF="/etc/hostapd/hostapd.conf"** Gibt den Pfad zur Hauptkonfigurationsdatei von hostapd an, die beim Start geladen wird.

Diese Einstellung gewährleistet, dass der WLAN-Access-Point beim Booten des Systems automatisch mit der definierten Konfiguration gestartet wird.

3.7 STARTSKRIPT FÜR DAS WLAN (STARTUPWIFI.SH)

Zusätzlich wurde ein Skript mit dem Namen **startupWIFI.sh** erstellt. Dieses befindet sich im Verzeichnis **/scripts/startupWIFI.sh**.

Das Skript dient dazu, die notwendigen Netzwerkdienste und Einstellungen für den WLAN-Access-Point zu starten. Nötig wurde es, weil aktuelle Versionen von Raspberry Pi OS die früher übliche Datei /etc/dhcpcd.conf nicht mehr verwenden. Die feste IP-Adresse muss daher auf anderem Weg gesetzt werden.

```
#!/bin/bash
ip addr add 192.168.10.1/24 dev wlan0
rfkill unblock wifi
ip link set wlan0 up
systemctl restart hostapd
systemctl restart dnsmasq
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

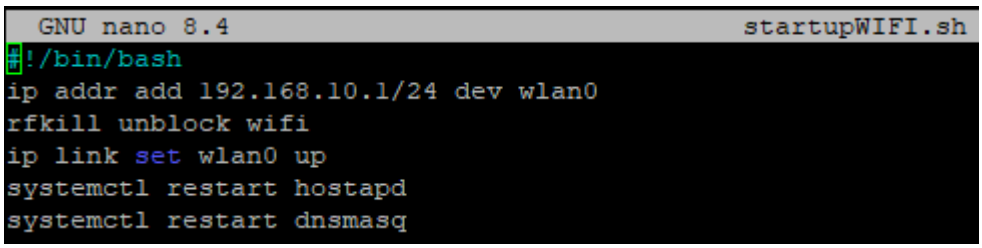
- **#!/bin/bash** Gibt an, mit welchem Programm die Datei ausgeführt werden soll. Fehlt diese Zeile, bricht das Skript mit der Meldung *Exec format error* ab.
- **ip addr add 192.168.10.1/24 dev wlan0** Weist dem WLAN-Interface wlan0 die statische IP-Adresse 192.168.10.1 mit dem Subnetz /24 (255.255.255.0) zu. Diese Adresse dient als Gateway für das WLAN-Netzwerk.
- **rfkill unblock wifi** Hebt eine mögliche Software- oder Hardware-Sperre für das WLAN-Modul auf.
- **ip link set wlan0 up** Aktiviert das Netzwerkinterface wlan0.

- **systemctl restart hostapd** Startet den hostapd-Dienst neu, damit das WLAN ausgestrahlt wird.
- **systemctl restart dnsmasq** Startet den Dienst dnsmasq neu, der DHCP- und DNS-Funktionen für das WLAN bereitstellt.

Das Skript muss ausführbar gemacht werden:

```
sudo chmod +x /scripts/startupWIFI.sh
```

Beim Ausführen von Hand ist `sudo` nötig, weil alle Befehle Systemrechte benötigen. Ohne sudo erscheint die Meldung *RTNETLINK answers: Operation not permitted*.



3.8 AUTOMATISCHER START DES SKRIPTS

Damit das Skript nach jedem Neustart von selbst ausgeführt wird, haben wir dafür einen eigenen Dienst angelegt. Die Datei liegt unter **/etc/systemd/system/startup-wifi.service**.

```
[Unit]
Description=WLAN Access Point Setup
After=network.target

[Service]
Type=oneshot
ExecStart=/scripts/startupWIFI.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

- **After=network.target** Der Dienst startet erst, wenn das Netzwerk grundsätzlich bereit ist.
- **Type=oneshot** Das Skript läuft einmal durch und beendet sich danach.
- **RemainAfterExit=yes** Der Dienst gilt trotzdem weiterhin als aktiv. Der Status *active (exited)* ist hier also der richtige Zustand.
- **WantedBy=multi-user.target** Der Dienst wird beim normalen Systemstart mitgestartet.

Aktiviert wurde er mit:

```
sudo systemctl daemon-reload
sudo systemctl enable startup-wifi.service
sudo systemctl enable hostapd
sudo systemctl enable dnsmasq
```

3.9 INTERNET-WEITERGABE INS WLAN

Über das WLAN allein kommt kein Internet herein. Der Raspberry Pi bezieht es über das LAN-Kabel und gibt es an die verbundenen Geräte weiter. Dafür sind zwei Dinge notwendig.

Zuerst muss die Weiterleitung von Datenpaketen erlaubt werden. Dazu haben wir die Datei **/etc/sysctl.d/99-wetterstation.conf** angelegt:

```
net.ipv4.ip_forward=1
```

Wir haben diesen Wert bewusst in eine eigene Datei geschrieben. Zuvor hatten wir ihn nur mit `sysctl -w` gesetzt, wodurch er nach jedem Neustart wieder verloren ging.

Anschließend wurden drei Regeln gesetzt, die den Verkehr zwischen wlan0 und eth0 übersetzen:

```
sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
sudo iptables -A FORWARD -i eth0 -o wlan0 -m state --state RELATED,ESTABLISHED -j ACCEPT
sudo iptables -A FORWARD -i wlan0 -o eth0 -j ACCEPT
sudo netfilter-persistent save
sudo systemctl enable netfilter-persistent
```

Bedeutung der Regeln:

- **POSTROUTING -o eth0 -j MASQUERADE** Pakete aus dem WLAN treten nach außen unter der Adresse des Raspberry Pi auf.
- **FORWARD -i eth0 -o wlan0 mit RELATED,ESTABLISHED** Antworten aus dem Internet dürfen zurück ins WLAN.
- **FORWARD -i wlan0 -o eth0** Anfragen aus dem WLAN dürfen ins Internet.
- **netfilter-persistent save** Speichert die Regeln dauerhaft, damit sie beim Booten wieder geladen werden.

Ob alles aktiv ist, prüft man mit:

```
cat /proc/sys/net/ipv4/ip_forward
sudo iptables -t nat -L POSTROUTING -n -v
```

Der erste Befehl muss den Wert 1 ausgeben.

4. Sensorik – BME680

4.1 DER SENSOR

Wir verwenden einen BME680 auf einem CJMCU-680-Breakout-Board. Der Sensor misst vier Größen gleichzeitig: Temperatur, Luftfeuchte, Luftdruck und einen Gaswiderstand, aus dem sich die Luftqualität ableiten lässt.

Die Verbindung zum Raspberry Pi erfolgt über den I2C-Bus. Dafür werden nur vier Kabel benötigt, die beiden übrigen Anschlüsse des Boards (SDO und CS) bleiben frei, sie werden nur für den SPI-Betrieb gebraucht.

4.2 VERKABELUNG

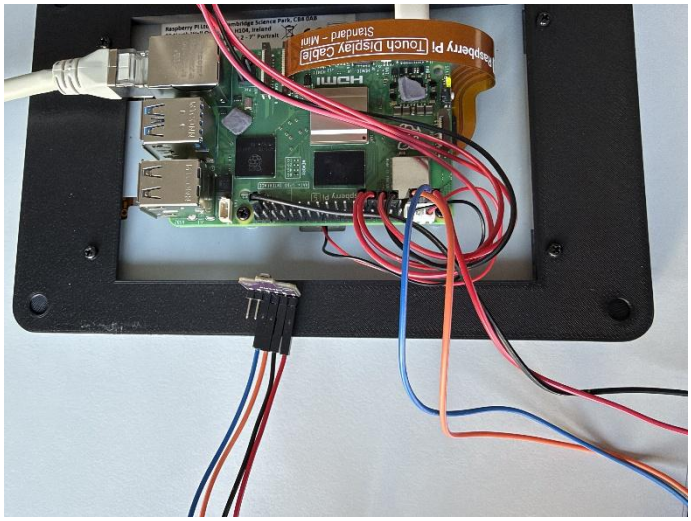
Vor dem Anschließen muss der Raspberry Pi ausgeschaltet und vom Strom getrennt werden.

BME680	RASPBERRY PI	ERKLÄRUNG
VCC	Pin 1 (3,3 V)	Stromversorgung
GND	Pin 39 (GND)	Masse
SCL	Pin 5 (GPIO 3)	Taktleitung
SDA	Pin 3 (GPIO 2)	Datenleitung

Wichtig ist, dass VCC an 3,3 V angeschlossen wird und nicht an die 5 V von Pin 2 oder Pin 4. Diese Boards haben meist keinen Spannungsregler, der Sensor selbst verträgt maximal 3,6 V.

Zu beachten ist außerdem die Reihenfolge der Anschlüsse am Board: Dort kommt **SCL vor SDA**, also umgekehrt zur üblichen Darstellung. Werden die beiden vertauscht, wird der Sensor später nicht gefunden.

Die Pins 2, 4 und 6 sind bei uns bereits durch das Touch Display belegt, deshalb haben wir für die Masse Pin 39 verwendet. Alle GND-Pins sind intern verbunden, es spielt also keine Rolle, welcher davon genommen wird.



4.3 I2C AKTIVIEREN

Standardmäßig ist der I2C-Bus abgeschaltet und muss zuerst aktiviert werden:

```
sudo raspi-config
```

Dort wählt man *Interface Options*, danach *I2C* und bestätigt mit *Yes*. Anschließend ist ein Neustart nötig:

```
sudo reboot
```

Ob der Sensor erkannt wird, prüft man mit:

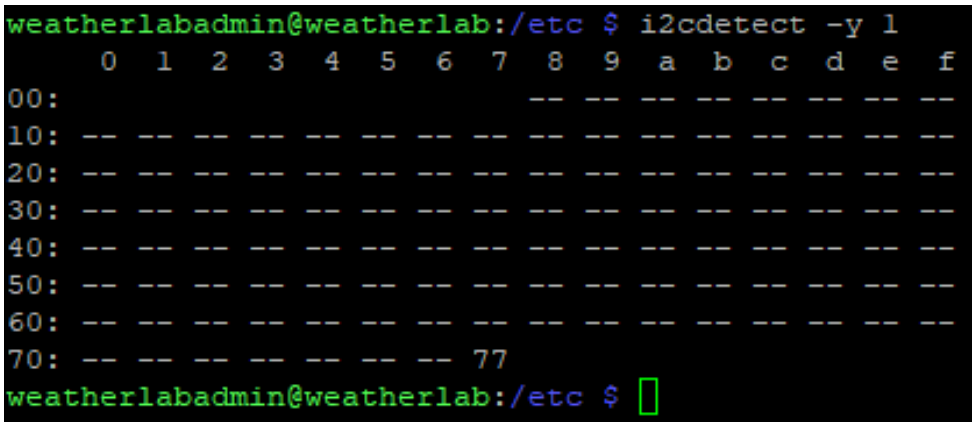
```
sudo apt install -y i2c-tools
i2cdetect -y 1
```

In der ausgegebenen Tabelle muss die Adresse **76** oder **77** erscheinen. Bei unserem Board meldet sich der Sensor unter **0x77**, weil der SDO-Anschluss intern auf 3,3 V liegt. Erscheint gar nichts, sind meistens SDA und SCL vertauscht.

Damit auch unser Programm später auf den Bus zugreifen darf, wurde der Benutzer der Gruppe i2c hinzugefügt:

```
sudo usermod -aG i2c weatherlabadmin
```

Diese Änderung wirkt erst nach einem Neustart.



4.4 PYTHON-BIBLIOTHEK

Für das Auslesen verwenden wir die Bibliothek bme680 in der Version 2.0.0, installiert in einer eigenen Python-Umgebung:

```
cd ~/pi-dashboard
python3 -m venv venv
venv/bin/pip install flask requests bme680 smbus2 psutil gpiozero
```

Beim Einrichten sind wir hier auf ein Problem gestoßen: Unser Programm verwendete zunächst die Konstante ENABLE_GAS_MEASUREMENT, die in dieser Version jedoch **ENABLE_GAS_MEAS** heißt. Dadurch brach die Initialisierung mit der Meldung *module*

'bme680' has no attribute 'ENABLE_GAS_MEASUREMENT' ab, und es wurden überhaupt keine Werte angezeigt.

Gelöst haben wir das so, dass beide Schreibweisen akzeptiert werden:

```
ENABLE_GAS = getattr(bme680, "ENABLE_GAS_MEASUREMENT", getattr(bme680, "ENABLE_GAS_MEAS", 1))
```

4.5 KONFIGURATION DES SENSORS

Im Programm wird der Sensor folgendermaßen eingestellt:

```
sensor.set_humidity_oversample(bme680.OS_2X)
sensor.set_pressure_oversample(bme680.OS_4X)
sensor.set_temperature_oversample(bme680.OS_8X)
sensor.set_filter(bme680.FILTER_SIZE_3)
sensor.set_gas_status(ENABLE_GAS)
sensor.set_gas_heater_temperature(320)
sensor.set_gas_heater_duration(150)
sensor.select_gas_heater_profile(0)
```

Bedeutung der Einstellungen:

- **Oversample-Werte** Geben an, wie oft intern gemessen und gemittelt wird. Höhere Werte bedeuten genauere, aber langsamere Messungen.
- **set_filter(FILTER_SIZE_3)** Glättet kurzfristige Schwankungen beim Luftdruck und bei der Temperatur.
- **set_gas_heater_temperature(320)** Heizt die Gasmesszelle auf 320 Grad Celsius auf. Nur in diesem aufgeheizten Zustand liefert sie brauchbare Werte.
- **set_gas_heater_duration(150)** Legt die Heizdauer in Millisekunden fest.

Da der Sensor zum Messen der Gase aufheizt, erwärmt er sich bei häufiger Messung selbst. Die angezeigte Temperatur kann dadurch um etwa 0,5 bis 1 Grad zu hoch liegen. Über die Einstellung SENSOR_INTERVAL lässt sich das Messintervall bei Bedarf von einer auf drei Sekunden erhöhen.

Außerdem braucht der Gassensor nach dem ersten Einschalten etwa 24 bis 48 Stunden Einbrennzeit, bis die Werte stabil sind.

5. Status-LEDs für die Luftqualität

5.1 ZWECK

Damit man die Luftqualität auch aus einiger Entfernung erkennt, ohne auf den Bildschirm zu schauen, haben wir drei LEDs ergänzt. Sie zeigen denselben Wert an, der auch auf der Website steht.

LUFTQUALITÄT AUF DER WEBSITE	LED
Sehr gut, Gut	grün
Mittel	gelb
Schlecht, Sehr schlecht	rot

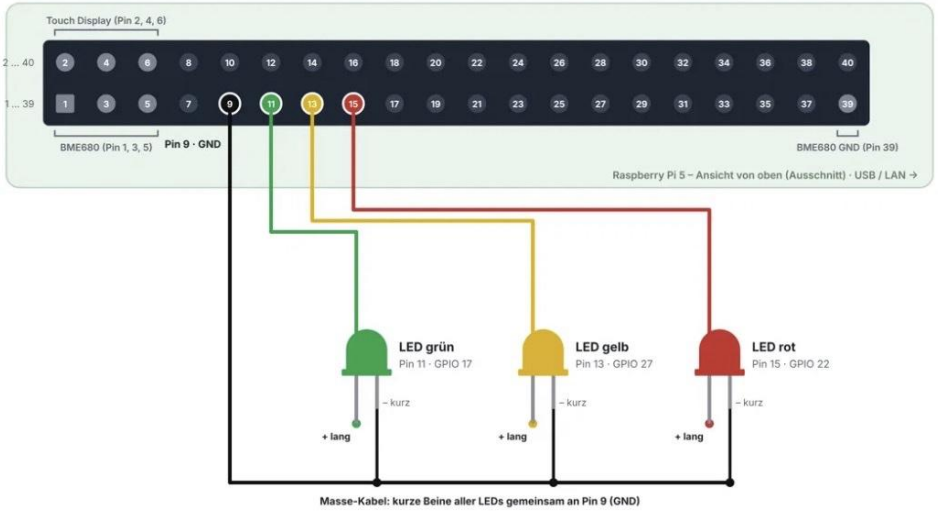
Es leuchtet immer genau eine LED, die beiden anderen werden beim Wechsel ausgeschaltet. Solange noch kein Messwert vorliegt, bleiben alle drei dunkel.

5.2 VERKABELUNG

Die LEDs sind ohne Vorwiderstand direkt an die GPIO-Pins angeschlossen. Das lange Bein (Plus) kommt an den GPIO-Pin, das kurze Bein (Minus) an die gemeinsame Masse.

LED	LANGES BEIN (+) AN	KURZES BEIN AN
grün	Pin 11 (GPIO 17)	gemeinsame Masse (PIN 9)
gelb	Pin 13 (GPIO 27)	gemeinsame Masse (PIN 9)
rot	Pin 15 (GPIO 22)	gemeinsame Masse (PIN 9)

Als gemeinsame Masse haben wir Pin 9 verwendet. Alle drei kurzen Beine dürfen auf denselben GND-Pin gelegt werden, da alle Masseanschlüsse intern miteinander verbunden sind. Auf dem Steckbrett führt dafür ein einziges Kabel von Pin 9 zur Minus-Schiene am Rand.



5.3 BENÖTIGTE BIBLIOTHEKEN

Angesteuert werden die LEDs über die Bibliothek gpiozero. Diese benötigt auf dem Raspberry Pi 5 zusätzlich lgpio.

```
sudo apt install -y python3-lgpio
cd ~/pi-dashboard
echo "include-system-site-packages = true" >> venv/pyvenv.cfg
venv/bin/pip install gpiozero
```

Der Versuch, lgpio direkt über pip zu installieren, schlug bei uns fehl, weil das Paket aus dem Quellcode gebaut wird und dafür das Programm swig benötigt. Deshalb verwenden wir das fertige Systempaket und erlauben der Python-Umgebung, Systempakete mitzubenutzen.

5.4 TEST DER LEDS

Vor dem Einbau in das Programm haben wir die Verkabelung einzeln getestet:

```
cd ~/pi-dashboard
venv/bin/python3 -c "
from gpiozero import LED
from time import sleep
for pin, name in ((17,'grün'), (27,'gelb'), (22,'rot')):
    l = LED(pin); l.on(); print(name, 'an'); sleep(1.5); l.off(); l.close()"
```

Nacheinander leuchtet jede LED für anderthalb Sekunden auf. Leuchtet eine nicht, ist meist die Polung vertauscht und die LED muss umgedreht werden.

5.5 EINBINDUNG IN DAS PROGRAMM

Im Programm werden die LEDs beim Start angelegt und bei jeder Messung aktualisiert:

```
def update_leds(quality):
    if not leds:
        return
    farbe = {"Sehr gut": "gruen", "Gut": "gruen", "Mittel": "gelb",
             "Schlecht": "rot", "Sehr schlecht": "rot"}.get(quality)
    for name, led in leds.items():
        led.on() if name == farbe else led.off()
```

Die Zuordnung greift auf dasselbe Ergebnis zu, das auch an die Website geliefert wird. Dadurch können Anzeige und LEDs nicht auseinanderlaufen.

Die Pins lassen sich über die Konfigurationsdatei ändern:

```
LED_ENABLED = True
LED_PIN_GRUEN = 17
LED_PIN_GELB = 27
LED_PIN_ROT = 22
```

Ist gpiozero nicht vorhanden oder tritt beim Zugriff ein Fehler auf, läuft das Programm ohne LEDs weiter. Die Wetterstation funktioniert also auch dann, wenn die LEDs abgesteckt sind.

6. Webanwendung

6.1 AUFBAU DES PROJEKTORDNERS

Das gesamte Projekt liegt in einem einzigen Ordner. Wer diesen sichert, hat alles beisammen.

```
/home/weatherlabadmin/pi-dashboard/
├── app.py              Programm: Sensor, Wetter, Datenbank, Schnittstellen
├── config.py           Einstellungen, API-Key, Standort, Bot-Token, Chat-IDs
├── history.db          Messverlauf, wird automatisch angelegt
├── telegram_bot.py     Telegram-Bot (eigener Dienst)
├── telegram_state.json aktive Alarme, letzter Tagesbericht
├── telegram-setup.sh   Einrichtung des Bots
├── venv/               Python-Umgebung mit den installierten Paketen
├── templates/
│   └── index.html      Aufbau der Website
└── static/
    ├── style.css       Aussehen der Website
    └── app.js          Logik im Browser
```

6.2 DIE DATEI CONFIG.PY

In dieser Datei stehen alle Werte, die man anpassen möchte. Am Programmcode selbst muss dadurch nichts geändert werden.

```
OPENWEATHER_API_KEY = "..."
LAT = 48.3069
LON = 14.2858

SENSOR_INTERVAL = 1
WEATHER_INTERVAL = 60
FORECAST_INTERVAL = 1800

SEA_LEVEL_PRESSURE_HPA = 1013.25

GAS_QUALITY_THRESHOLDS = {
    "sehr_gut": 130000,
    "gut": 90000,
    "mittel": 50000,
    "schlecht": 20000,
}

HISTORY_INTERVAL = 60
HISTORY_DAYS = 7
NIGHT_START = 22
NIGHT_END = 6
IDLE_RETURN_SECONDS = 90

LED_ENABLED = True
LED_PIN_GRUEN = 17
LED_PIN_GELB = 27
LED_PIN_ROT = 22
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

- **OPENWEATHER_API_KEY** Der kostenlose Zugangsschlüssel von OpenWeather. Er besteht aus 32 Zeichen. Ein neu erstellter Schlüssel wird bis zu zwei Stunden lang abgelehnt, bis er freigeschaltet ist.

- **LAT / LON** Die Koordinaten des Standorts, bei uns Linz. Sie gelten für das Außenwetter, die Vorhersage und die Karten.
- **SENSOR_INTERVAL** Abstand zwischen zwei Messungen des BME680 in Sekunden.
- **WEATHER_INTERVAL** Abstand der Abfragen bei OpenWeather. 60 Sekunden ergeben etwa 1.440 Abfragen pro Tag und liegen im kostenlosen Kontingent.
- **FORECAST_INTERVAL** Abstand der Abfragen für die 5-Tage-Vorhersage, hier alle 30 Minuten.
- **SEA_LEVEL_PRESSURE_HPA** Referenzdruck auf Meereshöhe, wird für die Höhenberechnung benötigt.
- **GAS_QUALITY_THRESHOLDS** Schwellenwerte, ab denen die Luftqualität in die jeweilige Stufe eingeteilt wird.
- **HISTORY_INTERVAL / HISTORY_DAYS** Legen fest, wie oft ein Messpunkt gespeichert und wie lange er aufbewahrt wird.
- **NIGHT_START / NIGHT_END** Zeitraum, in dem das Display abgedunkelt wird.
- **IDLE_RETURN_SECONDS** Zeit ohne Berührung, nach der die Anzeige zur Startseite zurückkehrt.
- **LED_ENABLED und die LED-Pins** Schalten die Status-LEDs ein und legen die verwendeten GPIO-Nummern fest.

6.3 DIE DATEI APP.PY

Das Programm ist in Python geschrieben und verwendet Flask als Webserver. Beim Start werden vier Schleifen als eigene Threads gestartet, die im Hintergrund dauerhaft laufen:

```
if __name__ == "__main__":
    init_db()
    init_leds()
    threading.Thread(target=sensor_loop, daemon=True).start()
    threading.Thread(target=weather_loop, daemon=True).start()
    threading.Thread(target=forecast_loop, daemon=True).start()
    threading.Thread(target=history_loop, daemon=True).start()
    app.run(host="0.0.0.0", port=5000, threaded=True)
```

Aufgaben der einzelnen Funktionen:

FUNKTION	AUFGABE
init_db()	legt die Tabelle in der Datenbank an, falls sie fehlt
init_leds()	meldet die drei Status-LEDs an
sensor_loop()	liest den BME680 jede Sekunde aus
weather_loop()	holt alle 60 Sekunden das aktuelle Wetter
forecast_loop()	holt alle 30 Minuten die 5-Tage-Vorhersage
history_loop()	speichert jede Minute einen Messpunkt in der Datenbank
get_pi_stats()	liest CPU-Temperatur, Auslastung, RAM und Speicher aus

Die Schleifen sammeln die Daten also im Hintergrund, während die Schnittstellen nur ausgeben, was bereits vorliegt. Dadurch antwortet die Website sofort, auch wenn OpenWeather gerade langsam reagiert. Damit nicht gleichzeitig geschrieben und gelesen wird, sind die gemeinsamen Daten über sogenannte Locks abgesichert.

Findet das Programm den Sensor beim Start nicht, versucht es alle 10 Sekunden erneut, statt sich zu beenden. Dadurch läuft die Website auch ohne angeschlossenen Sensor weiter.

6.4 SCHNITTSTELLEN

Die Website holt sich die Werte über mehrere Adressen, die reine Daten im JSON-Format zurückgeben. Sie lassen sich auch im Terminal aufrufen, was bei der Fehlersuche hilft.

ADRESSE	LIEFERT
/api/indoor	Innenwerte, Trend, Tages-Min/Max, Luftdrucktendenz
/api/outdoor	Außenwetter, Luftdruck, Sonnenauf- und -untergang
/api/forecast	5-Tage-Vorhersage
/api/history	Messverlauf für 6 Stunden, 24 Stunden oder 7 Tage
/api/pi-stats	CPU, RAM, Speicher und Laufzeit des Raspberry Pi
/api/location	Standort und Einstellungen für den Nachtmodus

Beispielaufruf:

```
curl -s http://localhost/api/indoor
```

6.5 DATENBANK

Für den Messverlauf verwenden wir SQLite. Dabei handelt es sich um eine dateibasierte Datenbank, die keinen eigenen Serverdienst benötigt und bereits in Python enthalten ist. Die gesamte Datenbank liegt in der Datei **history.db** im Projektordner.

```
CREATE TABLE readings (
    ts       INTEGER PRIMARY KEY,   -- Zeitstempel
    in_temp  REAL,                  -- Innentemperatur
    in_hum   REAL,                  -- Innenfeuchte
    out_temp REAL,                  -- Außentemperatur
    out_hum  REAL,                  -- Außenfeuchte
    pressure REAL                   -- Luftdruck
);
```

Jede Minute wird eine Zeile geschrieben, das ergibt 1.440 Zeilen pro Tag. Einträge, die älter als acht Tage sind, löscht das Programm automatisch. Bei sieben Tagen Verlauf sind das rund 10.000 Zeilen und etwa ein bis zwei Megabyte.

Für das Diagramm werden die Minutenwerte per SQL mit AVG und GROUP BY zu größeren Zeitabschnitten zusammengefasst, damit bei sieben Tagen nicht 10.000 Punkte gezeichnet werden müssen.

Den Inhalt kann man sich bei Bedarf ansehen:

```
sudo apt install -y sqlite3
sqlite3 -header -column ~/pi-dashboard/history.db "SELECT datetime(ts,'unixepoch','localtime'),
in_temp, out_temp FROM readings ORDER BY ts DESC LIMIT 5;"
```

7. Berechnete Werte

Nicht alles, was auf der Website steht, wird direkt gemessen. Einige Werte berechnet das Programm aus den Messdaten.

7.1 LUFTQUALITÄT

Der BME680 liefert keine fertige Luftqualität, sondern einen elektrischen Widerstand in Ohm. Ein hoher Widerstand bedeutet saubere Luft, ein niedriger viele Gase wie Ausdünstungen, Kochdunst oder verbrauchte Atemluft. Daraus bilden wir fünf Stufen:

STUFE	GASWIDERSTAND
Sehr gut	ab 130.000 Ω
Gut	ab 90.000 Ω
Mittel	ab 50.000 Ω
Schlecht	ab 20.000 Ω
Sehr schlecht	darunter

```
def gas_quality_label(gas_resistance):
    t = config.GAS_QUALITY_THRESHOLDS
    if gas_resistance >= t["sehr_gut"]:
        return "Sehr gut"
    if gas_resistance >= t["gut"]:
        return "Gut"
    if gas_resistance >= t["mittel"]:
        return "Mittel"
    if gas_resistance >= t["schlecht"]:
        return "Schlecht"
    return "Sehr schlecht"
```

Wichtig zu wissen ist, dass diese Werte relativ und nicht geeicht sind. Der Sensor sagt also nicht, wie viel CO_2 in der Luft ist, sondern nur, ob die Luft besser oder schlechter als sonst in diesem Raum ist. Die Schwellenwerte sollten nach der Einbrennzeit an den jeweiligen Raum angepasst werden.

Ein Wert wird außerdem erst übernommen, wenn der Sensor `heat_stable` meldet, der Heizer also seine Betriebstemperatur erreicht hat. Das dauert nach jedem Start ein bis zwei Minuten.

7.2 LUFTDRUCK UND TENDENZ

Zunächst hatten wir aus dem Luftdruck die Höhe über dem Meeresspiegel berechnet:

```
def compute_altitude(pressure_hpa):
    return round(44330 * (1 - (pressure_hpa / config.SEA_LEVEL_PRESSURE_HPA) ** (1 / 5.255)), 1)
```

In der Praxis hat sich das jedoch als unpraktisch erwiesen. Der Luftdruck ändert sich wetterbedingt täglich um mehrere Hektopascal, wodurch die angezeigte Höhe laufend um einige Dutzend Meter schwankte, obwohl das Gerät gar nicht bewegt wurde. Eine Anpassung des Referenzdrucks stimmt immer nur am Tag der Einstellung.

Deshalb haben wir uns entschieden, stattdessen den Luftdruck selbst anzuzeigen, zusammen mit seiner Tendenz über die letzten drei Stunden. Genau das ist die eigentliche Wetterinformation, früher haben Barometer mit Angaben wie *Schön*, *Veränderlich* und *Regen* dasselbe gemeint.

ÄNDERUNG IN 3 STUNDEN	ANZEIGE
ab +3 hPa	stark steigend
ab +1 hPa	steigend
zwischen −1 und +1 hPa	gleichbleibend
ab −1 hPa	fallend
ab −3 hPa	stark fallend

Fallender Luftdruck deutet auf eine Wetterverschlechterung hin, steigender auf eine Besserung. Für die Berechnung wird der aktuelle Wert mit dem Mittelwert aus der Datenbank verglichen, der etwa drei Stunden zurückliegt.

7.3 LÜFTUNGSEMPFEHLUNG

Aus Innen- und Außenwerten leitet die Website einen Hinweis ab, ob sich Lüften gerade lohnt.

BEDINGUNG	ANZEIGE
Luftqualität Schlecht oder Sehr schlecht	Bitte lüften, die Luftqualität ist schlecht
Feuchte über 60 % und draußen trockenere Luft	Lüften empfohlen, draußen ist die Luft trockener
Feuchte über 60 %, draußen nicht trockener	Luft feucht, Lüften bringt gerade wenig
Feuchte unter 35 %	Luft sehr trocken
Temperatur über 26 °C	Raum sehr warm
Temperatur unter 18 °C	Raum kühl
sonst	Raumklima angenehm

Der Vergleich erfolgt dabei nicht über die Prozentwerte, sondern über die absolute Luftfeuchte in Gramm pro Kubikmeter. Das ist wichtig, weil kalte Außenluft mit 80 % tatsächlich weniger Wasser enthalten kann als warme Innenluft mit 60 %. Berechnet wird sie mit der Magnus-Formel:

```
function absoluteHumidity(tempC, relHum) {
  const sat = 6.112 * Math.exp((17.67 * tempC) / (tempC + 243.5));
  return (sat * relHum * 2.1674) / (273.15 + tempC);
}
```

7.4 WEITERE ABGELEITETE WERTE

WERT	BERECHNUNG
Tages-Min/Max	kleinster und größter Wert der Innentemperatur seit Mitternacht, per SQL aus der Datenbank

WERT	BERECHNUNG
Temperaturtrend	Vergleich des aktuellen Werts mit dem Mittelwert von vor 30 Minuten, Schwelle 0,3 °C
Gefühlte Temperatur	wird direkt von OpenWeather geliefert
Windgeschwindigkeit	OpenWeather liefert Meter pro Sekunde, wir rechnen mit Faktor 3,6 in km/h um

8. Weboberfläche

8.1 AUFBAU

Die Website besteht aus drei Dateien, die zusammenarbeiten:

DATEI	AUFGABE
templates/index.html	legt fest, welche Elemente es gibt und wo sie stehen
static/style.css	bestimmt das Aussehen, also Farben, Größen und Abstände
static/app.js	holt die Werte vom Programm und trägt sie in die Elemente ein

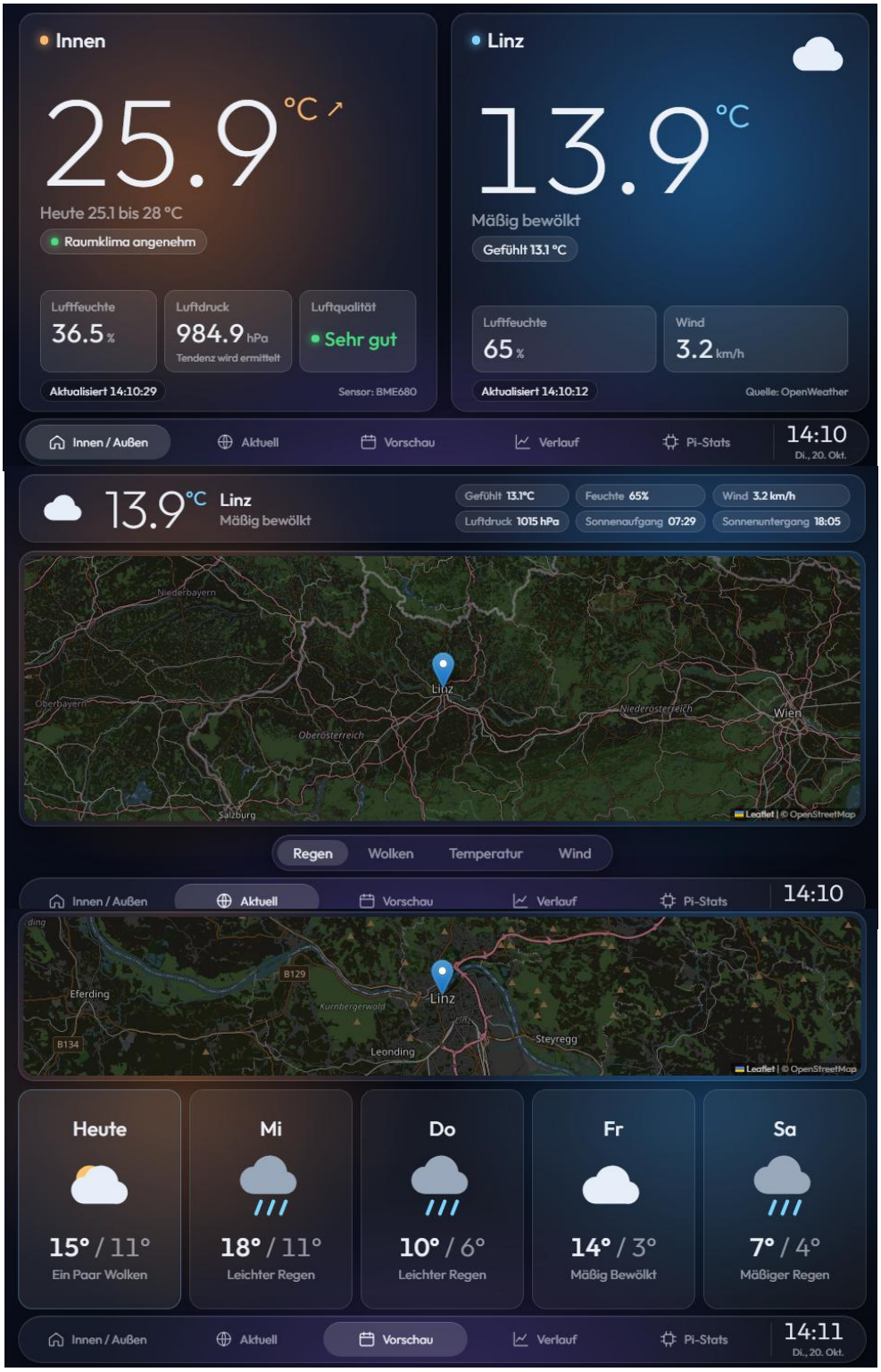
Jedes Feld im HTML hat eine eindeutige Kennung, zum Beispiel `id="in-temp"` für die Innentemperatur. Über genau diese Namen findet app.js die Stelle, an die der Wert gehört. Die Zeilen mit `{{ url_for(...) }}` sind Platzhalter von Flask, dort werden beim Ausliefern die richtigen Pfade zu style.css und app.js eingesetzt.

8.2 DIE FÜNF SEITEN

Über die Navigationsleiste am unteren Rand lässt sich zwischen fünf Seiten wechseln. Die Seiten sind alle im HTML enthalten, es wird jeweils nur eine davon angezeigt. Dadurch muss beim Umschalten nichts nachgeladen werden.

SEITE	INHALT
Innen / Außen	Innentemperatur mit Trend und Tages-Min/Max, Luftfeuchte, Luftdruck mit Tendenz, Luftqualität, Lüftungsempfehlung und daneben das aktuelle Außenwetter
Aktuell	Wetter im Detail mit gefühlter Temperatur, Luftdruck, Sonnenauf- und -untergang sowie eine Karte, auf der Regen, Wolken, Temperatur oder Wind eingeblendet werden können
Vorschau	Standortkarte und die Vorhersage für die nächsten fünf Tage
Verlauf	Diagramm der Innen- und Außentemperatur sowie der Luftfeuchte, umschaltbar zwischen 6 Stunden, 24 Stunden und 7 Tagen
Pi-Stats	CPU-Temperatur, CPU-Auslastung, Arbeitsspeicher, Speicherplatz und Laufzeit des Raspberry Pi

Zusätzlich befindet sich rechts in der Navigationsleiste eine Uhr mit Datum, die auf jeder Seite sichtbar ist.


Innen
25.9 °C ↗
Heute 25.1 bis 28 °C
Raumklima angenehm
Luftfeuchte
36.5 %
Luftdruck
984.9 hPa
Tendenz wird ermittelt
Luftqualität
Sehr gut
Aktualisiert 14:10:29
Sensor: BME680
Linz
13.9 °C
Mäßig bewölkt
Gefühlt 13.1 °C
Luftfeuchte
65 %
Wind
3.2 km/h
Aktualisiert 14:10:12
Quelle: OpenWeather
Innen / Außen
Aktuell
Vorschau
Verlauf
Pi-Stats
14:10
Di., 20. Okt.
13.9°C Linz
Mäßig bewölkt
Gefühlt 13.1°C
Feuchte 65%
Wind 3.2 km/h
Luftdruck 1015 hPa
Sonnenaufgang 07:29
Sonnenuntergang 18:05
Niederbayern
Oberbayern
Linz
Oberösterreich
Niederösterreich
Wien
Salzburg
Leaflet | © OpenStreetMap
Regen
Wolken
Temperatur
Wind
Innen / Außen
Aktuell
Vorschau
Verlauf
Pi-Stats
14:10
Eferding
B129
Kürnbergerwald
Linz
B134
Leonding
Steyregg
Leaflet | © OpenStreetMap
Heute
15° / 11°
Ein Paar Wolken
Mi
18° / 11°
Leichter Regen
Do
10° / 6°
Leichter Regen
Fr
14° / 3°
Mäßig Bewölkt
Sa
7° / 4°
Mäßiger Regen
Innen / Außen
Aktuell
Vorschau
Verlauf
Pi-Stats
14:11
Di., 20. Okt.



8.3 GESTALTUNG

Für das Aussehen haben wir uns für ein dunkles Design entschieden, weil das Display dauerhaft eingeschaltet ist und ein heller Hintergrund im Raum störend wirken würde. Die Kacheln sind halbtransparent gestaltet und liegen über farbigen Lichtflecken im Hintergrund, warm auf der Innenseite und kühl auf der Außenseite.

Alle Größen sind in den Einheiten `vw` und `vh` angegeben, also in Prozent der Bildschirmbreite und -höhe. Dadurch passt sich die Darstellung automatisch an, egal ob die Seite auf dem 7-Zoll-Display oder auf einem Handy geöffnet wird.

Die Wettersymbole haben wir selbst als SVG gezeichnet, statt die Bilder von OpenWeather zu laden. Dadurch passen sie zum Design und funktionieren auch dann, wenn gerade keine Internetverbindung besteht.

8.4 KARTENMATERIAL

Für die Karten verwenden wir die Bibliothek Leaflet zusammen mit Kacheln von OpenStreetMap. Die Wetter-Ebenen für Regen, Wolken, Temperatur und Wind kommen von OpenWeather.

Zunächst hatten wir dunkle Kacheln von CARTO eingebunden. Dieser Anbieter verlangt inzwischen jedoch einen eigenen Schlüssel, weshalb über der Karte dauerhaft der Schriftzug *API KEY REQUIRED* lag. Wir sind deshalb auf die normalen Kacheln von OpenStreetMap umgestiegen, die keinen Schlüssel benötigen, und färben sie per CSS dunkel ein:

```
.basemap-dark { filter: invert(1) hue-rotate(180deg) brightness(0.88) contrast(0.92)
saturate(0.6); }
```

8.5 AKTUALISIERUNG DER WERTE

Die Seite fragt die Werte in unterschiedlichen Abständen ab, je nachdem wie oft sie sich ändern:

ANGEZEIGTER WERT	ABSTAND IM BROWSER
Innenwerte	1 Sekunde
Außenwerte	10 Sekunden
Pi-Systemwerte	5 Sekunden
Uhr	1 Sekunde
Verlauf (bei geöffneter Seite)	60 Sekunden
Vorschau	30 Minuten

8.6 ZUSATZFUNKTIONEN FÜR DEN DAUERBETRIEB

Da das Display rund um die Uhr läuft, haben wir zwei Komfortfunktionen ergänzt:

- **Nachtmodus** Zwischen 22 und 6 Uhr wird die Anzeige abgedunkelt. Ein Antippen weckt sie für zwei Minuten. Über die Konfiguration lässt sich der Zeitraum ändern oder die Funktion ganz abschalten.
- **Rückkehr zur Startseite** Wurde 90 Sekunden lang nichts berührt, wechselt die Anzeige automatisch zurück auf die Seite *Innen / Außen*. So steht nach einem Blick auf eine andere Seite nicht dauerhaft die falsche Ansicht am Display.

9. Webserver und Autostart

9.1 DIENST FÜR DIE WEBANWENDUNG

Damit das Programm beim Einschalten von selbst startet und nach einem Absturz neu startet, haben wir dafür einen systemd-Dienst angelegt. Die Datei liegt unter **/etc/systemd/system/pi-dashboard.service**.

```
[Unit]
Description=Pi Dashboard (Flask)
After=network-online.target
Wants=network-online.target

[Service]
User=weatherlabadmin
WorkingDirectory=/home/weatherlabadmin/pi-dashboard
ExecStart=/home/weatherlabadmin/pi-dashboard/venv/bin/python3 /home/weatherlabadmin/pi-
dashboard/app.py
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

- **After / Wants=network-online.target** Der Dienst startet erst, wenn das Netzwerk bereit ist.
- **User=weatherlabadmin** Das Programm läuft unter dem normalen Benutzerkonto und nicht als root.
- **WorkingDirectory** Legt den Arbeitsordner fest. Dort wird auch die Datenbank history.db abgelegt.
- **ExecStart** Startet app.py mit dem Python aus unserer eigenen Umgebung.
- **Environment=PYTHONUNBUFFERED=1** Sorgt dafür, dass Meldungen des Programms sofort im Protokoll erscheinen. Ohne diese Zeile bleiben sie im Puffer hängen, was uns bei der Fehlersuche zunächst einige Zeit gekostet hat.
- **Restart=always, RestartSec=5** Nach einem Absturz wird der Dienst nach fünf Sekunden automatisch neu gestartet.

Aktiviert und gestartet wurde er mit:

```
sudo systemctl daemon-reload
sudo systemctl enable --now pi-dashboard.service
sudo systemctl status pi-dashboard.service
```

9.2 APACHE ALS REVERSE PROXY

Die Webanwendung selbst lauscht auf Port 5000. Man müsste also immer die Adresse mit der Portangabe eingeben. Damit die Seite unter der gewohnten Adresse ohne Port erreichbar ist, haben wir Apache davorgeschaltet. Apache nimmt die Anfragen auf Port 80 entgegen und leitet sie intern weiter.

Zuerst werden die benötigten Module aktiviert:

```
sudo apt install -y apache2
sudo a2enmod proxy proxy_http
```

Die Konfiguration liegt unter **/etc/apache2/sites-available/pi-dashboard.conf**:

```
<VirtualHost *:80>
    ServerName weatherlab

    ProxyPreserveHost On
    ProxyRequests Off

    ProxyPass / http://127.0.0.1:5000/
    ProxyPassReverse / http://127.0.0.1:5000/

    ErrorLog ${APACHE_LOG_DIR}/pi-dashboard-error.log
    CustomLog ${APACHE_LOG_DIR}/pi-dashboard-access.log combined
</VirtualHost>
```

Hier ist eine Auflistung Zeile für Zeile was jeder Eintrag bedeutet:

- **VirtualHost :80* Diese Konfiguration gilt für alle Anfragen, die auf Port 80 eintreffen.
- **ServerName weatherlab** Der Hostname, unter dem der Server angesprochen wird.
- **ProxyPreserveHost On** Gibt den ursprünglich angefragten Hostnamen an die Webanwendung weiter.
- **ProxyRequests Off** Verhindert, dass Apache als offener Proxy für fremde Seiten missbraucht werden kann.
- **ProxyPass / ProxyPassReverse** Leiten alle Anfragen an die Anwendung auf 127.0.0.1:5000 weiter und deren Antworten wieder zurück.
- **ErrorLog / CustomLog** Legen eigene Protokolldateien unter /var/log/apache2/ an.

Anschließend wird die Seite aktiviert und die Standardseite deaktiviert:

```
sudo a2ensite pi-dashboard.conf
sudo a2dissite 000-default.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

Der Befehl `configtest` muss *Syntax OK* melden. Bei einem Fehler läuft Apache mit der alten Konfiguration weiter, die Seite bleibt also erreichbar.

Meldet der Browser später den Fehler **503 Service Unavailable**, bedeutet das, dass Apache läuft, die Webanwendung dahinter aber nicht. In diesem Fall hilft ein Blick in das Protokoll des Dienstes.

9.3 KIOSK-MODUS AM TOUCH DISPLAY

Damit das Display nach dem Einschalten automatisch die Website im Vollbild anzeigt, haben wir den Browser in den Autostart eingetragen. Zuvor muss in `raspi-config` unter *System Options* die automatische Anmeldung am Desktop aktiviert werden.

Die Autostart-Datei liegt unter **~/.config/labwc/autostart**:

```
chromium --kiosk --noerrdialogs --disable-infobars --incognito --disable-session-crashed-bubble
http://localhost/ &
```

Bedeutung der Optionen:

- **--kiosk** Startet den Browser im Vollbild ohne Adressleiste und ohne Menüs.
- **--noerrdialogs** Unterdrückt Fehlerdialoge, die sonst über der Anzeige stehen bleiben würden.
- **--incognito** Es wird kein Verlauf gespeichert.
- **--disable-session-crashed-bubble** Unterdrückt die Meldung, dass Chromium nicht ordnungsgemäß beendet wurde. Diese würde nach jedem Stromausfall erscheinen.
- **http://localhost/** Die angezeigte Adresse. Da der Browser auf dem Raspberry Pi selbst läuft, genügt localhost.

Beenden lässt sich der Kiosk-Modus mit `Alt+F4` oder über SSH mit `pkill chromium`. Zum erneuten Starten über SSH ist die Angabe des Bildschirms nötig:

```
DISPLAY=:0 chromium --kiosk http://localhost/ &
```

9.4 ÜBERSICHT DER DIENSTE

Nach der vollständigen Einrichtung laufen folgende Dienste automatisch:

DIENST	AUFGABE
hostapd	sendet das WLAN aus
dnsmasq	vergibt IP-Adressen und löst Namen auf
startup-wifi	setzt die feste IP und startet die beiden oberen Dienste
pi-dashboard	die Webanwendung mit Sensor und Wetter
apache2	liefert die Website auf Port 80 aus
telegram-bot	Telegram-Bot mit Alarmen und Tagesbericht (Abschnitt 10.7)

Alle gemeinsam prüfen:

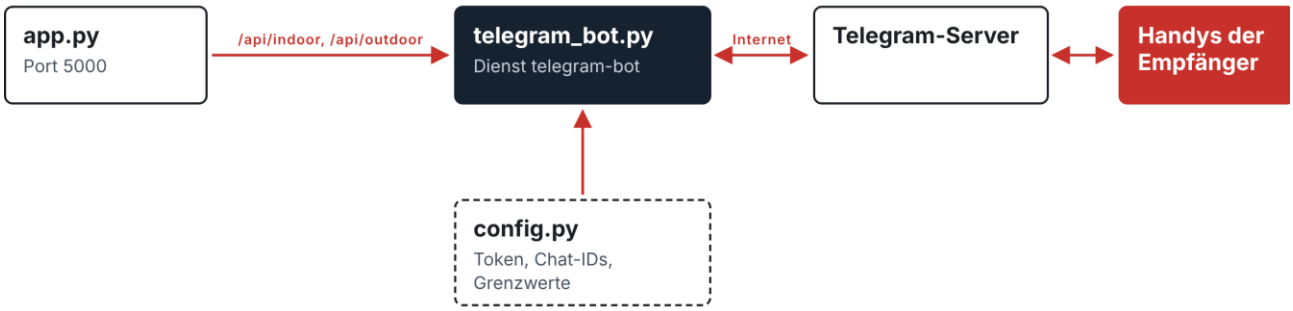
```
systemctl status hostapd dnsmasq startup-wifi pi-dashboard apache2 telegram-bot --no-pager |
grep -E "●|Active"
```

Jeder Dienst sollte *active (running)* melden. Bei startup-wifi ist *active (exited)* der richtige Zustand, weil das Skript einmal durchläuft und sich danach beendet.

10. Telegram-Bot

In der zweiten Projektphase haben wir die Wetterstation um einen Telegram-Bot erweitert. Er schickt Alarme und jeden Morgen einen Tagesbericht aufs Handy und beantwortet Befehle wie /wetter.

Der Bot läuft als eigener Dienst neben der Webanwendung. Er holt seine Werte über dieselben Schnittstellen wie die Website (`http://127.0.0.1:5000/api/...`). Dadurch mussten wir an der funktionierenden Messung in app.py nichts ändern.



Neue Dateien im Projektordner und im System:

DATEI	ZWECK
telegram_bot.py	der Bot: Befehle, Alarme, Tagesbericht
telegram_state.json	aktive Alarme, Datum des letzten Tagesberichts
config.py (ergänzt)	Token, freigeschaltete Chat-IDs, Grenzwerte
telegram-setup.sh	richtet den Bot ein und schaltet Chats frei
/etc/systemd/system/telegram-bot.service	startet den Bot beim Booten

10.1 BOT BEI TELEGRAM ANLEGEN

Ein Telegram-Bot wird über den offiziellen Bot @BotFather angelegt. Dort schreibt man `/newbot`, vergibt einen Anzeigenamen und einen Benutzernamen, der auf `bot` enden muss. BotFather antwortet mit dem Token, zum Beispiel:

```
7412589630:AAHf3kL9xQ2mZpR7vT1wYc8NbE4sJdU6oGi
```

Vor dem Doppelpunkt steht die ID des Bots, danach der geheime Teil. Wer den Token kennt, kann den Bot steuern. Er gehört deshalb nur in config.py und nicht auf Screenshots. Wird er doch einmal öffentlich, erzeugt `/revoke` bei BotFather einen neuen. Mit `/setuserpic` und `/setdescription` lassen sich außerdem Profilbild und Beschreibung des Bots ändern.

Der Token und alle Einstellungen des Bots stehen am Ende von **config.py**:

```
TELEGRAM_BOT_TOKEN = "7412589630:AAHf..."
TELEGRAM_CHAT_IDS = [123456789, 987654321]   # wer Nachrichten bekommt
TELEGRAM_DAILY_REPORT = "07:00"   # Tagesbericht um diese Uhrzeit ("" = aus)
TELEGRAM_ALERTS = True            # Alarme schicken
```

```
TELEGRAM_QUIET_START = "22:00"    # in dieser Zeit kommen Alarme lautlos
TELEGRAM_QUIET_END = "07:00"

ALERT_TEMP_HIGH = 27        # °C innen
ALERT_TEMP_LOW = 17         # °C innen
ALERT_HUMIDITY_HIGH = 65    # % innen
ALERT_HUMIDITY_LOW = 30     # % innen
ALERT_PRESSURE_DROP = -3    # hPa in 3 Stunden
ALERT_FROST_C = 0           # °C außen
ALERT_WIND_KMH = 50         # km/h außen
ALERT_SENSOR_TIMEOUT = 180  # Sekunden ohne neue Sensorwerte
```

Hier ist eine kurze Erklärung zu den einzelnen Werten:

- **TELEGRAM_BOT_TOKEN:** der Schlüssel von BotFather. Ohne ihn startet der Bot nicht.
- **TELEGRAM_CHAT_IDS:** Liste der Chats, die Nachrichten bekommen und Befehle nutzen dürfen (Abschnitt 10.4). Gruppen haben eine negative ID.
- **TELEGRAM_DAILY_REPORT:** Uhrzeit des Tagesberichts. Eine leere Angabe schaltet ihn ab.
- **TELEGRAM_QUIET_START / END:** In diesem Zeitraum kommen Alarme ohne Ton, damit nachts niemand geweckt wird.
- **ALERT_...:** die Grenzwerte, ab denen ein Alarm ausgelöst wird (Abschnitt 10.3).

Wie in app.py liest der Bot jeden Wert mit `getattr` und einem Standardwert. Fehlt eine Zeile in config.py, läuft er trotzdem. Nach einer Änderung muss der Bot mit `sudo systemctl restart telegram-bot` neu gestartet werden.

10.2 AUFBAU VON TELEGRAM_BOT.PY

Der Bot ist ein einzelnes Python-Programm ohne zusätzliche Pakete. Für die Verbindung zu Telegram reicht die Bibliothek requests, die für OpenWeather schon installiert war. Er spricht direkt mit der Telegram-Bot-API unter `https://api.telegram.org/bot<Token>/<Methode>`.

Neue Nachrichten holt der Bot mit **Long Polling** ab: Er fragt mit `getUpdates` nach, und Telegram hält die Anfrage bis zu 20 Sekunden offen, bis eine Nachricht kommt. So kommen Befehle fast sofort an, ohne dass der Bot im Sekundentakt nachfragen muss. Einen öffentlich erreichbaren Server braucht der Pi dafür nicht.

```
def main():
    me = tg("getMe")                        # Token prüfen
    tg("setMyCommands", commands=[...])     # Befehlsliste im Telegram-Menü
    state = load_state()
    watch = SensorWatch()
    broadcast("🟢 WeatherLab-Bot ist gestartet. /hilfe zeigt alle Befehle.")

    last_check = 0
    while True:
        # 1) Neue Nachrichten abholen (wartet bis zu 20 s)
        updates = tg_updates(state["offset"])
        for upd in updates:
            state["offset"] = upd["update_id"] + 1
            if "message" in upd:
                handle_message(upd["message"], state)

        # 2) Jede Minute Alarme und Tagesbericht prüfen
        if time.time() - last_check >= CHECK_INTERVAL:
            last_check = time.time()
```

```
            if CHAT_IDS:
                if ALERTS_ON:
                    check_alarms(state, watch)
                check_daily_report(state)
```

Hier ist eine kurze Erklärung zum Ablauf:

- **offset:** Jede Nachricht hat eine fortlaufende Nummer. Mit `offset = letzte Nummer + 1` bestätigt der Bot, dass er sie verarbeitet hat. Sonst würde Telegram sie nach einem Neustart erneut schicken. Der offset wird deshalb in telegram_state.json gespeichert.
- **check_alarms / check_daily_report:** laufen zwischen zwei Abfragen höchstens einmal pro Minute (`TELEGRAM_CHECK_INTERVAL`, Standard 60 s).
- **Fehler:** Ist Telegram oder das Internet nicht erreichbar, wartet der Bot 10 Sekunden und versucht es erneut. Im Protokoll wird der Token dabei durch `***` ersetzt.

Befehle:

BEFEHL	ANTWORT
/wetter	Innen- und Außenwerte mit Lüftungstipp
/innen	Messwerte des BME680 mit Trend, Tages-Min/Max und Luftdrucktendenz
/aussen	aktuelles Wetter von OpenWeather
/vorschau	5-Tage-Vorschau
/verlauf	Minimum und Maximum der letzten 24 Stunden
/alarme	aktive Alarme und alle Grenzwerte
/id	eigene Chat-ID
/hilfe	Liste aller Befehle

Die Antworten sind mit HTML formatiert (`parse_mode="HTML"`), zum Beispiel fette Werte. Alle Texte, die aus den Messdaten kommen, laufen vorher durch `html.escape`, damit ein Zeichen wie `<` in einer Wetterbeschreibung die Formatierung nicht zerstört. Zahlen werden mit Komma ausgegeben (22,4 °C).

10.3 ALARME UND TAGESBERICHT

Jede Minute holt der Bot `/api/indoor` und `/api/outdoor` und prüft alle Alarme. Zu jedem Alarm gehören zwei Bedingungen: eine zum Auslösen und eine zum Zurücksetzen. Der Abstand dazwischen heißt **Hysterese**. Ohne sie würde ein Wert, der genau an der Grenze pendelt (27,0 → 26,9 → 27,0 °C), jede Minute einen neuen Alarm und eine neue Entwarnung auslösen.

ALARM	QUELLE	AUSLÖSEN BEI	ENTWARNUNG BEI
Innen zu warm	BME680	≥ 27 °C	< 26 °C
Innen zu kalt	BME680	≤ 17 °C	> 18 °C
Luftfeuchte zu hoch	BME680	≥ 65 %	< 60 %
Luft zu trocken	BME680	≤ 30 %	> 35 %

ALARM	QUELLE	AUSLÖSEN BEI	ENTWARNUNG BEI
Schlechte Luft	BME680	Schlecht oder Sehr schlecht	Gut oder Sehr gut
Luftdruck fällt stark	BME680 + Verlauf	≤ −3 hPa in 3 h	> −1,5 hPa in 3 h
Station offline	BME680	180 s kein neuer Messwert	wieder neue Werte
Frost	OpenWeather	≤ 0 °C	> 1 °C
Starker Wind	OpenWeather	≥ 50 km/h	< 40 km/h
Gewitter	OpenWeather	Wettersymbol 11 (Gewitter)	anderes Wettersymbol

Die Werte in der Tabelle sind die Standardwerte aus config.py. Bei der Luftqualität bildet die Stufe „Mittel“ die Hysterese: Der Alarm bleibt bestehen, bis die Luft wieder „Gut“ ist. Beim Alarm „Luftfeuchte zu hoch“ hängt der Bot die Lüftungsempfehlung aus Abschnitt 7.3 an, also ob Lüften gerade hilft.

Die Alarme sind als Tabelle (Python-Wörterbuch) im Programm hinterlegt. Jeder Eintrag hat einen Namen, die beiden Bedingungen und die beiden Texte:

```
ALARMS = {
    "temp_hoch": {
        "name": "Innen zu warm",
        "an":  lambda i, o: _cmp(_v(i, "temperature"), lambda v: v >= TEMP_HIGH),
        "aus": lambda i, o: _cmp(_v(i, "temperature"), lambda v: v < TEMP_HIGH - 1),
        "text": lambda i, o: f"🔥 <b>Innen zu warm:</b> {num(i['temperature'], ' °C')}",
        "ok":   lambda i, o: f"☑ Innentemperatur wieder normal: {num(i['temperature'], '
°C')}",
    },
    ...
}

for key, alarm in ALARMS.items():
    if key not in active and alarm["an"](indoor, outdoor):
        active.add(key)
        broadcast(alarm["text"](indoor, outdoor), alarm=True)
    elif key in active and alarm["aus"](indoor, outdoor):
        active.discard(key)
        broadcast(alarm["ok"](indoor, outdoor))
```

`i` steht für die Innenwerte, `o` für die Außenwerte. Fehlt ein Messwert, liefert `_cmp` den Wert `None`, und der Zustand des Alarms bleibt unverändert. Ein kurzer Ausfall von OpenWeather löst also keine falsche Entwarnung aus. Welche Alarme gerade aktiv sind, steht in telegram_state.json. Nach einem Neustart schickt der Bot deshalb nicht alle Alarme noch einmal.

Station offline: Die Innenwerte tragen einen Zeitstempel (`timestamp`). Der Bot merkt sich, wann sich dieser zuletzt geändert hat. Bleibt er länger als 180 Sekunden gleich oder ist das Dashboard nicht erreichbar, kommt eine Warnung mit dem Befehl zur Fehlersuche.

Nachtruhe: Zwischen 22:00 und 07:00 Uhr verschickt der Bot Alarme mit `disable_notification`. Sie kommen an, aber ohne Ton. Entwarnungen und der Tagesbericht kommen immer normal mit Ton.

Tagesbericht: Um 07:00 Uhr schickt der Bot Innenwerte, Außenwerte, Lüftungstipp und die Vorschau. War der Pi um diese Uhrzeit aus, holt er den Bericht innerhalb der nächsten Stunde nach. Das Datum des letzten Berichts steht in telegram_state.json, damit er nur einmal pro Tag kommt.

10.4 EMPFÄNGER FREISCHALTEN

Nicht jeder, der den Bot in Telegram findet, soll Messwerte und Alarme bekommen. Der Bot antwortet deshalb nur Chats, deren ID in config.py unter `TELEGRAM_CHAT_IDS` steht:

```
TELEGRAM_CHAT_IDS = [123456789, 987654321, -1002233445566]
```

Jeder Telegram-Nutzer und jede Gruppe hat eine feste Chat-ID. Gruppen-IDs beginnen mit einem Minus. Schreibt ein unbekannter Chat dem Bot `/start`, antwortet er mit dessen Chat-ID und dem Hinweis, wie man sie einträgt. Auf alle anderen Befehle antwortet er nur mit „Dieser Chat ist nicht freigeschaltet“.

```
if command in ("start", "id") and chat_id not in CHAT_IDS:
    send(chat_id, f"👋 Hallo! Deine Chat-ID ist <code>{chat_id}</code>.\n"
                  f"Trag sie in <code>config.py</code> bei TELEGRAM_CHAT_IDS ein "
                  f"und starte den Bot neu:\n"
                  f"<code>sudo systemctl restart telegram-bot</code>")
    log("Unbekannter Chat meldet sich:", chat_id)
    return
if chat_id not in CHAT_IDS:
    send(chat_id, "⛔ Dieser Chat ist nicht freigeschaltet. /id zeigt deine Chat-ID.")
    return
```

Eine weitere Person, zum Beispiel eine Lehrkraft, lässt sich auf zwei Wegen freischalten:

1. **Mit dem Einrichtungsskript:** `bash ~/pi-dashboard/telegram-setup.sh` ausführen, den vorhandenen Token mit Enter bestätigen und die Person dem Bot `/start` schreiben lassen, während das Skript wartet. Das Skript ergänzt die ID und startet den Bot neu (Abschnitt 10.6). Bereits eingetragene IDs bleiben erhalten.
2. **Von Hand:** Die Person schreibt dem Bot `/start` und schickt uns die angezeigte Chat-ID. Alternativ steht sie im Protokoll unter „Unbekannter Chat meldet sich“. Die Zahl wird mit Komma in die Liste in config.py eingetragen und der Bot neu gestartet:

```
sudo journalctl -u telegram-bot | grep "Unbekannter Chat"
nano ~/pi-dashboard/config.py
sudo systemctl restart telegram-bot
```

Nach dem Neustart bekommt jeder eingetragene Chat die Nachricht „WeatherLab-Bot ist gestartet“. Daran sieht man sofort, ob das Eintragen geklappt hat.

10.5 DIENST TELEGRAM-BOT

Damit der Bot beim Booten startet und nach einem Absturz neu startet, läuft er als eigener systemd-Dienst. Pfad: **/etc/systemd/system/telegram-bot.service**

```
[Unit]
Description=WeatherLab Telegram-Bot
After=network-online.target pi-dashboard.service
```

```
Wants=network-online.target

[Service]
User=weatherlabadmin
WorkingDirectory=/home/weatherlabadmin/pi-dashboard
ExecStart=/home/weatherlabadmin/pi-dashboard/venv/bin/python3 /home/weatherlabadmin/pi-
dashboard/telegram_bot.py
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Hier ist eine kurze Erklärung zu den Einträgen:

- **After=... pi-dashboard.service:** Der Bot startet nach der Webanwendung, weil er seine Werte von ihr holt.
- **User=weatherlabadmin:** Der Bot läuft ohne root-Rechte.
- **ExecStart:** startet den Bot mit dem Python aus der virtuellen Umgebung, in der requests installiert ist.
- **PYTHONUNBUFFERED=1:** Ausgaben erscheinen sofort im Protokoll. Ohne diese Zeile hält Python sie zurück, und `journalctl` zeigt lange nichts an.
- **Restart=always / RestartSec=10:** Nach einem Absturz startet der Bot nach 10 Sekunden neu.

Aktiviert und geprüft wird der Dienst so:

```
sudo systemctl daemon-reload
sudo systemctl enable --now telegram-bot
sudo journalctl -u telegram-bot -f          # Protokoll live mitlesen
```

Beim Start schreibt der Bot zum Beispiel `Bot @weatherlab_linz_bot gestartet, freigeschaltete Chats: [123456789]` ins Protokoll und schickt allen Empfängern „WeatherLab-Bot ist gestartet“. Daran sieht man auch, wenn der Pi neu gestartet wurde.

10.6 EINRICHTUNG MIT TELEGRAM-SETUP.SH

Eingerichtet wird der Bot mit dem Skript **telegram-setup.sh**. Es enthält telegram_bot.py und legt die Datei selbst an, sodass nur eine Datei auf den Pi kopiert werden muss. Übertragen lässt sie sich zum Beispiel mit FileZilla (Protokoll SFTP, Port 22, Benutzer weatherlabadmin) und in den Projektordner gelegt. Gestartet wird das Skript als normaler Benutzer, nicht mit sudo:

```
bash ~/pi-dashboard/telegram-setup.sh
```

Das Skript arbeitet diese Schritte ab:

1. **telegram_bot.py anlegen** und mit `py_compile` auf Fehler prüfen.
2. **Laufenden Bot anhalten**, damit er die /start-Nachricht im nächsten Schritt nicht abfängt.
3. **Token abfragen** und mit `getMe` bei Telegram prüfen. Ist schon ein Token eingetragen, kann er mit Enter übernommen werden.

4. **Auf /start warten:** Bis zu 2 Minuten fragt das Skript bei Telegram nach neuen Nachrichten und merkt sich die Chat-IDs. Ältere Nachrichten werden vorher übersprungen.
5. **config.py ergänzen:** Vorher entsteht eine Sicherung `config.py.bak-telegram`. Fehlt der Telegram-Abschnitt, wird er mit den Standardwerten angehängt. Neue Chat-IDs kommen zur Liste dazu, vorhandene bleiben. Ist config.py danach fehlerhaft, spielt das Skript die Sicherung zurück.
6. **Dienst einrichten:** telegram-bot.service schreiben (Abschnitt 10.5), aktivieren und starten. Zum Schluss zeigt das Skript die letzten Zeilen des Protokolls.

Das Skript entfernt außerdem Windows-Zeilenenden selbst. Diese entstehen leicht beim Kopieren von einem Windows-PC und hatten bei startupWIFI.sh den Fehler „Exec format error“ verursacht. Es kann beliebig oft ausgeführt werden, zum Beispiel um weitere Chats freizuschalten oder einen neuen Token einzutragen.

10.7 TEST OHNE ECHTE TELEGRAM-VERBINDUNG

Bevor wir die Erweiterung auf den Pi gespielt haben, haben wir sie auf einem Rechner ohne Sensor und ohne Telegram getestet. Dafür kann man in config.py zwei Adressen umstellen, die sonst ihre Standardwerte haben:

```
TELEGRAM_API_URL = "http://127.0.0.1:8765"   # statt https://api.telegram.org
DASHBOARD_URL = "http://127.0.0.1:8765"      # statt http://127.0.0.1:5000
```

Unter dieser Adresse lief ein kleiner Python-Testserver. Er lieferte feste Messwerte wie `/api/indoor` und spielte gleichzeitig Telegram: Er beantwortete `getMe` und `getUpdates` und merkte sich jede `sendMessage`-Nachricht. So ließ sich prüfen, was der Bot verschicken würde.

TEST	ERGEBNIS
alle Befehle von einem freigeschalteten Chat	richtige Antworten, Zahlen mit Komma
/start von einem unbekannten Chat	Antwort mit der Chat-ID, keine Messwerte
Grenzwerte überschritten	je ein Alarm, beim zweiten Durchlauf kein doppelter
Wert knapp unter der Grenze (Hysterese)	keine Entwarnung
Werte wieder normal	Entwarnung für jeden Alarm
Sensor-Zeitstempel 16 min unverändert	Warnung „Wetterstation meldet sich nicht“
Dashboard nicht erreichbar	Meldung „Wetterstation ist gerade nicht erreichbar“
Tagesbericht zweimal aufgerufen	nur ein Bericht
Einrichtungsskript mit Testserver	Token geprüft, Chat-ID eingetragen, alte Nachricht übersprungen

11. Betrieb, Wartung und Fehlerbehebung

11.1 ÄNDERUNGEN ÜBERNEHMEN

Je nachdem, welche Datei geändert wurde, sind unterschiedliche Schritte nötig:

GEÄNDERTE DATEI	NÖTIG DANACH
config.py oder app.py	sudo systemctl restart pi-dashboard
index.html, style.css, app.js	im Browser Strg+F5
pi-dashboard.service	sudo systemctl daemon-reload, danach restart
hostapd.conf	sudo systemctl restart hostapd
dnsmasq.conf	sudo systemctl restart dnsmasq, Gerät neu verbinden
Apache-Konfiguration	configtest, danach sudo systemctl reload apache2

11.2 NÜTZLICHE BEFEHLE

```
journalctl -u pi-dashboard -f                    # Protokoll live mitlesen
journalctl -xeu pi-dashboard --no-pager | tail -30
journalctl -u telegram-bot -f                    # Protokoll des Bots
curl -s http://localhost/api/indoor              # Innenwerte prüfen
curl -s http://localhost/api/outdoor             # Außenwerte prüfen
i2cdetect -y 1                                   # ist der Sensor sichtbar?
ip addr show wlan0                               # hat wlan0 die feste IP?
nmcli device status                              # ist wlan0 unmanaged?
```

11.3 SICHERUNG

Der wichtigste Teil ist der Projektordner. Vom Windows-PC aus lässt er sich so kopieren:

```
scp -r weatherlabadmin@192.168.10.1:~/pi-dashboard C:\Backup\
```

Die Systemdateien aus Kapitel 3 lassen sich zusätzlich sichern:

```
sudo tar czf ~/konfig-backup.tar.gz /etc/hostapd/hostapd.conf /etc/dnsmasq.conf /etc/dnsmasq.d/
/etc/NetworkManager/conf.d/unmanaged.conf /scripts/startupWIFI.sh /etc/systemd/system/startup-
wifi.service /etc/systemd/system/pi-dashboard.service /etc/apache2/sites-available/pi-
dashboard.conf /etc/iptables/rules.v4
```

11.4 AUFGETRETENE PROBLEME UND IHRE LÖSUNG

Während der Einrichtung sind einige Probleme aufgetreten, die wir hier samt Lösung festhalten, weil sie beim Nachbauen erfahrungsgemäß wieder auftreten.

PROBLEM	URSACHE	LÖSUNG
Unit hostapd.service is masked	hostapd ist nach der Installation gesperrt	sudo systemctl unmask hostapd
hostapd startet nicht, unknown configuration	unvollständige Zeilen ieee80211n und ieee80211d	Zeilen entfernt, Prüfung mit sudo hostapd -dd

PROBLEM	URSACHE	LÖSUNG
item		
NetworkManager startet nicht mehr	fehlender Abschnitt [keyfile] und falsches Trennzeichen in unmanaged.conf	Datei korrigiert, danach systemctl reset-failed
Gerät bekommt keine IP-Adresse	wlan0 hatte keine feste Adresse	startupWIFI.sh ausführen bzw. als Dienst einrichten
Verbinden dauert sehr lange	nicht erreichbarer DNS-Server in dnsmasq.conf eingetragen	durch 1.1.1.1 und 8.8.8.8 ersetzt
WLAN vorhanden, aber kein Internet	ip_forward wurde nach jedem Neustart zurückgesetzt	Wert dauerhaft in /etc/sysctl.d/99-wetterstation.conf eingetragen
Exec format error beim Skript	Windows-Zeilenumbrüche, fehlende erste Zeile	dos2unix und #!/bin/bash ergänzt
Keine Sensorwerte, nur Striche	Konstante ENABLE_GAS_MEASUREMENT existiert in dieser Version nicht	beide Schreibweisen abgefangen
Außenwerte bleiben leer, Invalid API key	neuer Schlüssel noch nicht freigeschaltet	bis zu zwei Stunden warten
503 Service Unavailable	Apache läuft, die Webanwendung nicht	Protokoll prüfen, Syntaxfehler in app.py beheben
lgpio lässt sich nicht installieren	Paket wird aus dem Quellcode gebaut, swig fehlt	Systempaket python3-lgpio verwendet

11.5 HINWEISE FÜR DEN DAUERBETRIEB

- Der Gassensor benötigt nach dem ersten Einschalten 24 bis 48 Stunden, bis die Werte für die Luftqualität stabil sind. Die Schwellenwerte sollten danach an den Raum angepasst werden.
- Das Verlaufsdiagramm ist nach einer Neuinstallation zunächst leer. Die ersten Linien erscheinen nach etwa zehn Minuten, die vollständige 24-Stunden-Ansicht nach einem Tag.
- Die Luftdrucktendenz steht in den ersten drei Stunden auf *Tendenz wird ermittelt*, weil der Vergleichswert erst gesammelt werden muss.
- Nach einem Neustart dauert es ein bis zwei Minuten, bis die Luftqualität und damit auch die Status-LEDs wieder einen Wert anzeigen.

11.6 FAZIT

Das Projekt verbindet mehrere Bereiche miteinander: Netzwerktechnik beim Access Point, Elektronik beim Sensor und den LEDs, Programmierung bei der Webanwendung und Serveradministration bei Apache und den Diensten.

Besonders aufschlussreich war für uns, dass die meisten Fehler nicht im eigentlichen Programmcode lagen, sondern in Kleinigkeiten der Konfiguration: ein fehlender Doppelpunkt, eine abgeschnittene Zeile, ein unsichtbares Zeichen am Zeilenende oder eine

Einstellung, die nach dem Neustart wieder verloren ging. Das systematische Prüfen mit den Protokollen und den Testbefehlen hat sich dabei als der schnellste Weg zur Lösung erwiesen.

Das fertige System läuft eigenständig: Nach dem Einschalten startet der Access Point, die Messung beginnt, die Website wird ausgeliefert und das Display zeigt die Werte an, ohne dass jemand etwas eingeben muss.

Mit dem Telegram-Bot ist die Station auch aus der Ferne nutzbar: Sie meldet sich von selbst, wenn es drinnen zu warm oder zu feucht wird oder draußen Frost, Sturm oder ein Gewitter droht.

12. Anhang: Quellcode app.py

Dieser Anhang enthält den vollständigen Programmcode der Datei **app.py**. Zur besseren Lesbarkeit ist er in die einzelnen Abschnitte unterteilt, jeweils mit einer Erklärung davor.

12.1 KOPF UND EINSTELLUNGEN

Zu Beginn werden die benötigten Bibliotheken eingebunden. Der Import von config.py ist in eine Abfrage verpackt: Fehlt die Datei, wird das Programm mit einem verständlichen Hinweis beendet, statt mit einer unklaren Fehlermeldung abzustürzen. Auch der Import von gpiozero ist abgesichert, damit die Wetterstation ohne LEDs weiterläuft.

Die Einstellungen aus config.py werden mit `getattr` und einem Standardwert gelesen. Dadurch funktioniert das Programm auch mit einer älteren Konfigurationsdatei, in der die neueren Einträge noch fehlen.

```
"""
Pi Dashboard – Innen (BME680) & Außen (OpenWeather) im Webinterface.

Start:
    python3 app.py
Dann im Browser (bzw. Kiosk-Modus auf dem Touch Display):
    http://localhost:5000
"""

import os
import sqlite3
import subprocess
import threading
import time

import psutil
import requests
from flask import Flask, jsonify, render_template, request

try:
    import config
except ImportError:
    raise SystemExit(
        "config.py fehlt! Bitte config.py.example nach config.py kopieren "
        "und die Werte (API-Key, Standort) eintragen."
    )

import bme680

try:
    from gpiozero import LED
except Exception:  # Bibliothek fehlt oder kein GPIO vorhanden
    LED = None

app = Flask(__name__)

# Neue Einstellungen mit Standardwerten, damit eine ältere config.py weiter funktioniert
HISTORY_INTERVAL = getattr(config, "HISTORY_INTERVAL", 60)
HISTORY_DAYS = getattr(config, "HISTORY_DAYS", 7)
NIGHT_START = getattr(config, "NIGHT_START", 22)
NIGHT_END = getattr(config, "NIGHT_END", 6)
IDLE_RETURN_SECONDS = getattr(config, "IDLE_RETURN_SECONDS", 90)
LED_ENABLED = getattr(config, "LED_ENABLED", True)
LED_PIN_GRUEN = getattr(config, "LED_PIN_GRUEN", 17)
LED_PIN_GELB = getattr(config, "LED_PIN_GELB", 27)
```

```
LED_PIN_ROT = getattr(config, "LED_PIN_ROT", 22)

DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "history.db")
```

12.2 GEMEINSAME DATENSPEICHER

Hier werden drei Wörterbücher angelegt, in denen die zuletzt gemessenen Werte liegen. Die Hintergrundschleifen schreiben hinein, die Schnittstellen lesen daraus.

Zu jedem Speicher gehört ein sogenanntes Lock. Da mehrere Threads gleichzeitig laufen, verhindert es, dass gleichzeitig geschrieben und gelesen wird. Ohne diese Absicherung könnte die Website einen halb aktualisierten Datensatz erwischen.

```
sensor_lock = threading.Lock()
sensor_data = {
    "temperature": None,
    "humidity": None,
    "altitude": None,
    "pressure": None,
    "air_quality": None,
    "timestamp": None,
}

weather_lock = threading.Lock()
weather_data = {
    "temperature": None,
    "feels_like": None,
    "humidity": None,
    "pressure": None,
    "wind_speed": None,
    "description": None,
    "icon": None,
    "city": None,
    "sunrise": None,
    "sunset": None,
    "timestamp": None,
}

forecast_lock = threading.Lock()
forecast_data = {"days": [], "timestamp": None}
```

12.3 DATENBANK

`init_db` legt die Tabelle an, falls sie noch nicht existiert. Zusätzlich wird geprüft, ob die Spalte für den Luftdruck vorhanden ist. Diese Prüfung haben wir ergänzt, als wir den Luftdruck nachträglich aufgenommen haben, damit bestehende Datenbanken nicht gelöscht werden müssen.

`history_loop` läuft dauerhaft im Hintergrund und schreibt jede Minute eine Zeile. Danach werden Einträge gelöscht, die älter als acht Tage sind. Liegen weder Innen- noch Außenwerte vor, wird der Durchlauf übersprungen.

```
def db_connect():
    return sqlite3.connect(DB_PATH, timeout=5)


def init_db():
    with db_connect() as conn:
        conn.execute(
            "CREATE TABLE IF NOT EXISTS readings ("
            "ts INTEGER PRIMARY KEY, in_temp REAL, in_hum REAL, out_temp REAL, out_hum REAL)"
```

```
        )
        spalten = [z[1] for z in conn.execute("PRAGMA table_info(readings)")]
        if "pressure" not in spalten:
            conn.execute("ALTER TABLE readings ADD COLUMN pressure REAL")
            print("Datenbank um Spalte pressure erweitert")


def history_loop():
    """Speichert regelmäßig einen Messpunkt und löscht alte Einträge."""
    while True:
        time.sleep(HISTORY_INTERVAL)
        try:
            with sensor_lock:
                in_temp = sensor_data["temperature"]
                in_hum = sensor_data["humidity"]
                druck = sensor_data["pressure"]
            with weather_lock:
                out_temp = weather_data["temperature"]
                out_hum = weather_data["humidity"]
            if in_temp is None and out_temp is None:
                continue
            now = int(time.time())
            with db_connect() as conn:
                conn.execute(
                    "INSERT OR REPLACE INTO readings (ts, in_temp, in_hum, out_temp, out_hum,
pressure) "
                    "VALUES (?, ?, ?, ?, ?, ?)",
                    (now, in_temp, in_hum, out_temp, out_hum, druck),
                )
                conn.execute(
                    "DELETE FROM readings WHERE ts < ?", (now - (HISTORY_DAYS + 1) * 86400,)
                )
        except Exception as exc:
            print("Verlauf-Fehler:", exc)
```

12.4 AUSWERTUNG DES VERLAUFS

`pressure_trend` vergleicht den aktuellen Luftdruck mit dem Mittelwert von vor etwa drei Stunden. Dafür wird nicht ein einzelner alter Messwert genommen, sondern der Durchschnitt aus einer halben Stunde, damit einzelne Ausreißer das Ergebnis nicht verfälschen.

`indoor_extras` berechnet die Tageswerte und den Temperaturtrend. Für das Tagesminimum und -maximum wird ab Mitternacht gerechnet, der Trend ergibt sich aus dem Vergleich mit dem Wert von vor 30 Minuten.

```
PRESSURE_TREND_TEXT = {
    "stark_steigend": "stark steigend",
    "steigend": "steigend",
    "gleich": "gleichbleibend",
    "fallend": "fallend",
    "stark_fallend": "stark fallend",
}


def pressure_trend(current_pressure):
    """Vergleicht den aktuellen Luftdruck mit dem Wert von vor 3 Stunden."""
    if current_pressure is None:
        return None, None, None
    try:
        now = time.time()
        with db_connect() as conn:
            (frueher,) = conn.execute(
                "SELECT AVG(pressure) FROM readings WHERE ts BETWEEN ? AND ? AND pressure IS NOT
NULL",
```

```
                (now - 3.5 * 3600, now - 2.5 * 3600),
            ).fetchone()
        if frueher is None:
            return None, None, None
        diff = round(current_pressure - frueher, 1)
        if diff >= 3:
            stufe = "stark_steigend"
        elif diff >= 1:
            stufe = "steigend"
        elif diff <= -3:
            stufe = "stark_fallend"
        elif diff <= -1:
            stufe = "fallend"
        else:
            stufe = "gleich"
        return stufe, PRESSURE_TREND_TEXT[stufe], diff
    except Exception as exc:
        print("Drucktendenz-Fehler:", exc)
        return None, None, None


def indoor_extras(current_temp):
    """Tages-Min/Max und Trend der Innentemperatur aus dem Verlauf."""
    extras = {"today_min": None, "today_max": None, "trend": None}
    try:
        now = time.time()
        lt = time.localtime(now)
        midnight = time.mktime((lt.tm_year, lt.tm_mon, lt.tm_mday, 0, 0, 0, 0, 0, -1))
        with db_connect() as conn:
            mn, mx = conn.execute(
                "SELECT MIN(in_temp), MAX(in_temp) FROM readings WHERE ts >= ? AND in_temp IS
NOT NULL",
                (midnight,),
            ).fetchone()
            (past,) = conn.execute(
                "SELECT AVG(in_temp) FROM readings WHERE ts BETWEEN ? AND ? AND in_temp IS NOT
NULL",
                (now - 35 * 60, now - 25 * 60),
            ).fetchone()
        values = [v for v in (mn, mx, current_temp) if v is not None]
        if values:
            extras["today_min"] = round(min(values), 1)
            extras["today_max"] = round(max(values), 1)
        if past is not None and current_temp is not None:
            diff = current_temp - past
            extras["trend"] = "up" if diff > 0.3 else "down" if diff < -0.3 else "flat"
    except Exception as exc:
        print("Extras-Fehler:", exc)
    return extras
```

12.5 STATUS-LEDS

`init_leds` meldet die drei LEDs beim Start an. Schlägt das fehl, etwa weil die Bibliothek nicht vorhanden ist, wird die Liste geleert und das Programm läuft ohne LEDs weiter.

`update_leds` ordnet der Luftqualität eine Farbe zu und schaltet die übrigen LEDs ab. Dadurch leuchtet immer genau eine.

```
leds = {}


def init_leds():
    """Legt die drei LEDs an. Ohne gpiozero oder GPIO laeuft alles ohne LEDs weiter."""
    if not LED_ENABLED or LED is None:
        return
    try:
        leds["gruen"] = LED(LED_PIN_GRUEN)
```

```
        leds["gelb"] = LED(LED_PIN_GELB)
        leds["rot"] = LED(LED_PIN_ROT)
        print("Status-LEDs bereit: GPIO", LED_PIN_GRUEN, LED_PIN_GELB, LED_PIN_ROT)
    except Exception as exc:
        leds.clear()
        print("LED-Fehler, laeuft ohne LEDs weiter:", exc)


def update_leds(quality):
    """Sehr gut/Gut -> gruen, Mittel -> gelb, Schlecht/Sehr schlecht -> rot."""
    if not leds:
        return
    farbe = {
        "Sehr gut": "gruen",
        "Gut": "gruen",
        "Mittel": "gelb",
        "Schlecht": "rot",
        "Sehr schlecht": "rot",
    }.get(quality)
    try:
        for name, led in leds.items():
            led.on() if name == farbe else led.off()
    except Exception as exc:
        print("LED-Fehler:", exc)
```

12.6 SENSOR

`init_sensor` probiert beide möglichen I2C-Adressen durch. Unser Board meldet sich unter 0x77, andere unter 0x76. Schlagen beide fehl, wird ein verständlicher Fehler ausgelöst.

`sensor_loop` versucht beim Start so lange, den Sensor zu finden, bis es klappt. Danach wird im eingestellten Takt gemessen. Der Wert für die Luftqualität wird nur übernommen, wenn der Sensor `heat_stable` meldet, also der Heizer seine Temperatur erreicht hat.

```
def compute_altitude(pressure_hpa):
    """Höhe (m) aus Luftdruck über die internationale barometrische Höhenformel."""
    return round(
        44330 * (1 - (pressure_hpa / config.SEA_LEVEL_PRESSURE_HPA) ** (1 / 5.255)), 1
    )


def gas_quality_label(gas_resistance):
    """Ordnet den rohen BME680-Gaswiderstand (Ohm) einer Textkategorie zu."""
    if gas_resistance is None:
        return None
    t = config.GAS_QUALITY_THRESHOLDS
    if gas_resistance >= t["sehr_gut"]:
        return "Sehr gut"
    if gas_resistance >= t["gut"]:
        return "Gut"
    if gas_resistance >= t["mittel"]:
        return "Mittel"
    if gas_resistance >= t["schlecht"]:
        return "Schlecht"
    return "Sehr schlecht"


# Je nach Version der Bibliothek heisst die Konstante anders
ENABLE_GAS = getattr(bme680, "ENABLE_GAS_MEASUREMENT", getattr(bme680, "ENABLE_GAS_MEAS", 1))


def init_sensor():
    """BME680 initialisieren. Probiert beide I2C-Adressen (0x76 und 0x77)."""
    letzter_fehler = None
    sensor = None
    for adresse in (bme680.I2C_ADDR_PRIMARY, bme680.I2C_ADDR_SECONDARY):
```

```
        try:
            sensor = bme680.BME680(adresse)
            break
        except Exception as exc:
            letzter_fehler = exc
    if sensor is None:
        raise RuntimeError(f"BME680 auf 0x76 und 0x77 nicht erreichbar: {letzter_fehler}")

    sensor.set_humidity_oversample(bme680.OS_2X)
    sensor.set_pressure_oversample(bme680.OS_4X)
    sensor.set_temperature_oversample(bme680.OS_8X)
    sensor.set_filter(bme680.FILTER_SIZE_3)
    sensor.set_gas_status(ENABLE_GAS)
    sensor.set_gas_heater_temperature(320)
    sensor.set_gas_heater_duration(150)
    sensor.select_gas_heater_profile(0)
    return sensor


def sensor_loop():
    while True:
        try:
            sensor = init_sensor()
            break
        except Exception as exc:
            print("BME680 nicht gefunden, neuer Versuch in 10 s:", exc)
            time.sleep(10)
    while True:
        try:
            if sensor.get_sensor_data():
                with sensor_lock:
                    sensor_data["temperature"] = round(sensor.data.temperature, 1)
                    sensor_data["humidity"] = round(sensor.data.humidity, 1)
                    sensor_data["altitude"] = compute_altitude(sensor.data.pressure)
                    sensor_data["pressure"] = round(sensor.data.pressure, 1)
                    if sensor.data.heat_stable:
                        sensor_data["air_quality"] =
gas_quality_label(sensor.data.gas_resistance)
                        update_leds(sensor_data["air_quality"])
                    sensor_data["timestamp"] = time.strftime("%H:%M:%S")
        except Exception as exc:
            print("BME680-Fehler:", exc)
        time.sleep(config.SENSOR_INTERVAL)
```

12.7 ABFRAGE BEI OPENWEATHER

Beide Schleifen bauen die Adresse mit Koordinaten und Schlüssel zusammen und werten die Antwort aus. Der Parameter `units=metric` sorgt für Grad Celsius, `lang=de` für deutsche Wetterbeschreibungen.

Die Windgeschwindigkeit wird mit dem Faktor 3,6 von Meter pro Sekunde in Kilometer pro Stunde umgerechnet. Die Sonnenzeiten liefert OpenWeather als Zeitstempel, den wir in eine lesbare Uhrzeit umwandeln.

Bei der Vorhersage liefert OpenWeather Werte im Drei-Stunden-Takt. Wir gruppieren sie nach Datum, nehmen je Tag das Minimum und Maximum und bevorzugen für Symbol und Beschreibung den Eintrag der Mittagszeit, weil dieser das Tageswetter am besten beschreibt.

```
def weather_loop():
    while True:
        try:
            url = (
```

```
                "https://api.openweathermap.org/data/2.5/weather"
                f"?lat={config.LAT}&lon={config.LON}"
                f"&appid={config.OPENWEATHER_API_KEY}&units=metric&lang=de"
            )
            resp = requests.get(url, timeout=10)
            data = resp.json()

            if resp.status_code != 200:
                print("OpenWeather-Fehler:", data.get("message"))
            else:
                with weather_lock:
                    weather_data["temperature"] = round(data["main"]["temp"], 1)
                    weather_data["feels_like"] = round(data["main"]["feels_like"], 1)
                    weather_data["humidity"] = data["main"]["humidity"]
                    weather_data["pressure"] = data["main"].get("pressure")
                    weather_data["wind_speed"] = round(data["wind"]["speed"] * 3.6, 1)
                    weather_data["description"] = data["weather"][0]["description"]
                    weather_data["icon"] = data["weather"][0]["icon"]
                    weather_data["city"] = data["name"]
                    weather_data["sunrise"] = time.strftime("%H:%M",
time.localtime(data["sys"]["sunrise"]))
                    weather_data["sunset"] = time.strftime("%H:%M",
time.localtime(data["sys"]["sunset"]))
                    weather_data["timestamp"] = time.strftime("%H:%M:%S")
        except Exception as exc:
            print("Wetter-Fehler:", exc)
        time.sleep(config.WEATHER_INTERVAL)


def forecast_loop():
    """Holt die 5-Tage-Vorhersage (3h-Werte) und fasst sie zu Tageswerten zusammen."""
    while True:
        try:
            url = (
                "https://api.openweathermap.org/data/2.5/forecast"
                f"?lat={config.LAT}&lon={config.LON}"
                f"&appid={config.OPENWEATHER_API_KEY}&units=metric&lang=de"
            )
            resp = requests.get(url, timeout=10)
            data = resp.json()

            if resp.status_code != 200:
                print("Vorhersage-Fehler:", data.get("message"))
            else:
                days = {}
                for entry in data["list"]:
                    date, hour = entry["dt_txt"].split(" ")
                    day = days.setdefault(date, {"temps": [], "icon": None, "desc": None})
                    day["temps"].append(entry["main"]["temp"])
                    if hour == "12:00:00" or day["icon"] is None:
                        day["icon"] = entry["weather"][0]["icon"]
                        day["desc"] = entry["weather"][0]["description"]

                day_list = []
                for date, day in list(days.items())[:5]:
                    day_list.append(
                        {
                            "date": date,
                            "min": round(min(day["temps"]), 1),
                            "max": round(max(day["temps"]), 1),
                            "icon": day["icon"],
                            "description": day["desc"],
                        }
                    )
                with forecast_lock:
                    forecast_data["days"] = day_list
                    forecast_data["timestamp"] = time.strftime("%H:%M:%S")
        except Exception as exc:
            print("Vorhersage-Fehler:", exc)
        time.sleep(config.FORECAST_INTERVAL)
```

12.8 SYSTEMWERTE DES RASPBERRY PI

Die CPU-Temperatur wird über das Programm `vcgencmd` ausgelesen, das zu Raspberry Pi OS gehört. Falls das nicht funktioniert, wird ersatzweise die Systemdatei unter /sys gelesen. Die übrigen Werte liefert die Bibliothek psutil.

```
def get_pi_stats():
    """Liest CPU-Temperatur, Auslastung, RAM, Speicherplatz und Laufzeit aus."""
    temp = None
    try:
        out = subprocess.check_output(["vcgencmd", "measure_temp"]).decode()
        temp = float(out.strip().replace("temp=", "").replace("'C", ""))
    except Exception:
        try:
            with open("/sys/class/thermal/thermal_zone0/temp") as f:
                temp = round(int(f.read().strip()) / 1000, 1)
        except Exception:
            temp = None

    cpu_percent = psutil.cpu_percent(interval=0.3)
    mem = psutil.virtual_memory()
    disk = psutil.disk_usage("/")
    uptime_hours = round((time.time() - psutil.boot_time()) / 3600, 1)

    return {
        "cpu_temp": temp,
        "cpu_percent": cpu_percent,
        "ram_percent": mem.percent,
        "ram_used_mb": round(mem.used / 1024 / 1024),
        "ram_total_mb": round(mem.total / 1024 / 1024),
        "disk_percent": disk.percent,
        "disk_used_gb": round(disk.used / 1024**3, 1),
        "disk_total_gb": round(disk.total / 1024**3, 1),
        "uptime_hours": uptime_hours,
    }
```

12.9 SCHNITTSTELLEN UND PROGRAMMSTART

Jede Route gibt die gewünschten Daten als JSON zurück. Die meisten greifen einfach auf die bereits gesammelten Werte zu. Nur `api_history` rechnet selbst: Über SQL werden die Minutenwerte zu größeren Zeitabschnitten gemittelt, damit das Diagramm bei sieben Tagen nicht aus 10.000 Punkten besteht.

Am Ende werden die vier Hintergrundschleifen als Threads gestartet. Die Angabe `daemon=True` bewirkt, dass sie beim Beenden des Programms automatisch mit beendet werden.

```
@app.route("/")
def index():
    return render_template("index.html")


@app.route("/api/indoor")
def api_indoor():
    with sensor_lock:
        data = dict(sensor_data)
    data.update(indoor_extras(data["temperature"]))
    stufe, text, diff = pressure_trend(data["pressure"])
    data["pressure_trend"] = stufe
    data["pressure_trend_text"] = text
    data["pressure_change"] = diff
    return jsonify(data)
```

```
@app.route("/api/outdoor")
def api_outdoor():
    with weather_lock:
        return jsonify(weather_data)


@app.route("/api/forecast")
def api_forecast():
    with forecast_lock:
        return jsonify(forecast_data)


@app.route("/api/history")
def api_history():
    ranges = {"6h": (6 * 3600, 300), "24h": (86400, 900), "7d": (7 * 86400, 3600)}
    key = request.args.get("range", "24h")
    span, bucket = ranges.get(key, ranges["24h"])
    since = int(time.time()) - span
    with db_connect() as conn:
        rows = conn.execute(
            "SELECT (ts / ?) * ? AS b, AVG(in_temp), AVG(in_hum), AVG(out_temp) "
            "FROM readings WHERE ts >= ? GROUP BY b ORDER BY b",
            (bucket, bucket, since),
        ).fetchall()
    points = [
        [b, *(round(v, 1) if v is not None else None for v in vals)]
        for b, *vals in rows
    ]
    return jsonify({"range": key, "since": since, "points": points})


@app.route("/api/pi-stats")
def api_pi_stats():
    return jsonify(get_pi_stats())


@app.route("/api/location")
def api_location():
    return jsonify(
        {
            "lat": config.LAT,
            "lon": config.LON,
            "night_start": NIGHT_START,
            "night_end": NIGHT_END,
            "idle_return": IDLE_RETURN_SECONDS,
        }
    )


@app.route("/api/map-key")
def api_map_key():
    # Wird nur im eigenen Netzwerk aufgerufen, damit die Karte die Wetterebenen laden kann.
    return jsonify({"appid": config.OPENWEATHER_API_KEY})


if __name__ == "__main__":
    init_db()
    init_leds()
    threading.Thread(target=sensor_loop, daemon=True).start()
    threading.Thread(target=weather_loop, daemon=True).start()
    threading.Thread(target=forecast_loop, daemon=True).start()
    threading.Thread(target=history_loop, daemon=True).start()
    app.run(host="0.0.0.0", port=5000, threaded=True)
```

12.10 WIEDERKEHRENDE MUSTER IM CODE

Abschließend noch drei Dinge, die sich durch den gesamten Code ziehen und die wir bewusst so gelöst haben:

- **Fehler werden abgefangen, nicht weitergereicht.** Jede Schleife ist in einen try-except-Block verpackt. Fällt eine Abfrage aus, etwa weil das Internet kurz weg ist, wird eine Meldung ausgegeben und beim nächsten Durchlauf erneut versucht. Das Programm beendet sich dadurch nicht.
- **Sammeln und Ausgeben sind getrennt.** Die Schleifen schreiben in den Speicher, die Schnittstellen lesen nur daraus. Dadurch antwortet die Website in Millisekunden, auch wenn OpenWeather gerade mehrere Sekunden braucht.
- **Einstellungen stehen nie im Code.** Alles, was man ändern möchte, liegt in config.py. Dadurch muss am Programm selbst nichts angepasst werden, wenn sich zum Beispiel der Standort oder ein Schwellenwert ändert.